ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

KHOA KỸ THUẬT MÁY TÍNH




# BÁO CÁO TỔNG KẾT ĐỒ ÁN

## XÂY DỰNG MÔI TRƯỜNG KIỂM CHỨNG UVM CHO AXI4-FULL INTERCONNECT, RAM VÀ DMA

Sinh viên thực hiện: Hà Y Như Ý

Mã số sinh viên: 23521843

Giảng viên hướng dẫn: Nguyễn Thành Phát

# MỤC LỤC

**1. GIỚI THIỆU ĐỒ ÁN ..... 4**
1.1. Bối cảnh và lý do chọn đề tài ..... 4
1.2. Mục tiêu của đồ án ..... 4
1.3. Đối tượng kiểm chứng ..... 5
1.4. Công cụ và môi trường ..... 5
1.5. Tóm tắt kết quả ..... 5
1.6. Bố cục báo cáo ..... 6
**2. GIỚI THIỆU BUS AXI4 ..... 7**
2.1. Tổng quan ..... 7
2.2. Năm kênh và cơ chế bắt tay VALID/READY ..... 7
2.3. Burst, kích thước nhịp và strobe ..... 10
2.4. ID, thứ tự và các giao dịch đang chờ ..... 10
2.5. Mã đáp ứng và các tín hiệu phụ ..... 11
2.6. AXI4-Stream ..... 11
2.7. Những yêu cầu của chuẩn được đồ án kiểm tra ..... 11
**3. PHƯƠNG PHÁP KIỂM CHỨNG VÀ AXI4 VIP ..... 13**
3.1. Phương pháp kiểm chứng ..... 13
3.2. Kiến trúc của AXI4 VIP ..... 13
3.3. Hoạt động của các thành phần ..... 15
3.4. Hạ tầng chung của các testbench ..... 17
3.5. Tự kiểm chứng VIP ..... 18
**4. KIỂM CHỨNG BỘ NHỚ AXI_RAM ..... 20**
4.1. Cách hoạt động của axi_ram ..... 20
4.2. Testbench ..... 21
4.3. Các test đã thực hiện ..... 22
4.4. Nhận xét ..... 23
**5. KIỂM CHỨNG BỘ KẾT NỐI AXI_CROSSBAR ..... 24**
5.1. Cách hoạt động của axi_crossbar ..... 24
5.2. Cấu hình được kiểm chứng ..... 25
5.3. Testbench ..... 26

5.4. Các test đã thực hiện .....27
5.5. Các lỗi tìm được ở bộ kết nối .....29
**6. KIỂM CHỨNG CÁC ENGINE DMA VÀ HỆ THỐNG TÍCH HỢP .....31**
6.1. Cách hoạt động của ba engine DMA .....31
6.2. Testbench .....33
6.3. Các test đã thực hiện .....34
6.4. Các lỗi tìm được ở DMA .....35
6.5. Hệ thống tích hợp .....36
**7. TỔNG KẾT .....39**
7.1. Kết quả regression .....39
7.2. Chạy toàn bộ test trên mã gốc nguyên bản .....39
7.3. Độ phủ mã .....43
7.4. Chính sách loại trừ .....45
7.5. Độ phủ chức năng .....45
7.6. Tổng hợp các lỗi và giới hạn của thiết kế .....46
7.7. Đối chiếu với mục tiêu .....47
7.8. Hạn chế .....47
7.9. Hướng phát triển .....48
7.10. Kết luận .....48
**TÀI LIỆU THAM KHẢO .....49**

# 1. GIỚI THIỆU ĐỒ ÁN

## 1.1. Bối cảnh và lý do chọn đề tài

Trong một hệ thống trên chip (SoC), các khối xử lý, bộ nhớ và ngoại vi trao đổi dữ liệu với nhau qua bus. AMBA AXI4 của Arm [1] là chuẩn bus được dùng rộng rãi nhất hiện nay cho các kết nối cần băng thông cao. Hai loại khối hạ tầng xuất hiện trong hầu hết các SoC dùng AXI là bộ kết nối (interconnect), có nhiệm vụ định tuyến giao dịch từ nhiều master tới nhiều slave, và bộ truy cập bộ nhớ trực tiếp (DMA), có nhiệm vụ di chuyển các khối dữ liệu lớn mà không cần bộ xử lý can thiệp vào từng từ dữ liệu.

Vì mọi dữ liệu của hệ thống đều đi qua hai loại khối này, một lỗi nhỏ ở đây – mất một nhịp dữ liệu, trả lời nhầm master, treo sau một truy cập sai địa chỉ – sẽ làm hỏng cả hệ thống và rất khó truy vết ở mức hệ thống. Do đó chúng cần được kiểm chứng kỹ ở mức khối. Universal Verification Methodology (UVM) [3][4] là phương pháp chuẩn của công nghiệp cho việc này: kích thích ngẫu nhiên có ràng buộc, môi trường tự kiểm tra và đo độ phủ (coverage) để biết khi nào thì kiểm chứng đã đủ.

## 1.2. Mục tiêu của đồ án

- Xây dựng một **VIP (Verification IP) AXI4-Full** bằng UVM 1.2, không phụ thuộc vào thiết kế cụ thể nào, có thể đóng vai master, slave hoặc chỉ quan sát (passive), để dùng lại cho các thiết kế AXI khác sau này.
- Dùng VIP đó để kiểm chứng năm khối RTL: bộ nhớ axi_ram, bộ kết nối axi_crossbar và ba engine DMA là axi_dma_rd, axi_dma_wr, axi_cdma; sau đó kiểm chứng một hệ thống tích hợp gồm các khối này.
- Mọi test đều tự kiểm tra bằng scoreboard; phần lớn là test ngẫu nhiên chạy với nhiều seed; có thêm test có hướng (directed) cho những góc mà ngẫu nhiên khó chạm tới.
- Đạt độ phủ tổng (total coverage) trên 98% cho từng nhóm RAM, interconnect và DMA, với số liệu **trung thực**: báo cáo cả con số trước và sau khi loại trừ, mỗi mục loại trừ đều có lý do.
- Log của mỗi test phải cho biết test làm gì, kích thích ra sao, kiểm tra bằng cách nào và kết quả chi tiết, chứ không chỉ có bảng tổng kết của UVM.

## 1.3. Đối tượng kiểm chứng

RTL được lấy từ thư viện mã nguồn mở verilog-axi của Alex Forencich [6]. Khi đối chiếu với nhánh master của kho gốc, tám file của bộ kết nối giống hệt bản gốc; bốn file còn lại (axi_ram, axi_dma_rd, axi_dma_wr, axi_cdma) đã được chỉnh sửa trong project trước khi môi trường kiểm chứng được xây dựng (thêm tham số nạp nội dung ban đầu cho RAM, thêm trạng thái pipeline cho các engine DMA).

*Bảng 1.1. Các khối RTL được kiểm chứng*

| Nhóm | Khối RTL | Chức năng | So với mã gốc |
|---|---|---|---|
| RAM | axi_ram | Bộ nhớ có giao tiếp AXI4 slave | Đã chỉnh sửa (thêm INIT_FILE) |
| Interconnect | axi_crossbar, axi_crossbar_addr, axi_crossbar_rd, axi_crossbar_wr, axi_register_rd, axi_register_wr, arbiter, priority_encoder | Bộ kết nối không chặn (non-blocking) giữa nhiều master và nhiều slave | Giống hệt |
| DMA | axi_dma_rd | Đọc bộ nhớ qua AXI, xuất ra AXI-Stream | Đã chỉnh sửa |
| DMA | axi_dma_wr | Nhận AXI-Stream, ghi vào bộ nhớ qua AXI | Đã chỉnh sửa (thêm trạng thái pipeline) |
| DMA | axi_cdma | Sao chép bộ nhớ sang bộ nhớ qua AXI | Đã chỉnh sửa (thêm trạng thái pipeline) |

## 1.4. Công cụ và môi trường

- Trình mô phỏng QuestaSim 10.7c; thư viện UVM 1.2 đi kèm trình mô phỏng.
- Ngôn ngữ SystemVerilog cho VIP và testbench; RTL viết bằng Verilog.
- Các script PowerShell để biên dịch, chạy một test, chạy regression song song nhiều seed, gộp coverage và sinh báo cáo.

## 1.5. Tóm tắt kết quả

Regression cuối cùng gồm **340 lần mô phỏng** của 50 test: **326 PASS, 0 FAIL, 14 XFAIL** (XFAIL là các test minh hoạ lỗi đã biết của thiết kế, bắt buộc phải thất bại). Trong đó 312 lần chạy thuộc các test chạy với nhiều seed. Tổng số phép so sánh mà các scoreboard thực hiện trong các lần chạy PASS là khoảng 12,6 triệu.

Độ phủ mã tổng của RTL: RAM 94,95%, interconnect 94,39%, DMA 96,03% khi chưa loại trừ gì; sau khi loại trừ các mục chứng minh được là không thể đạt, cả ba nhóm đều vượt mốc 98% (chi tiết ở mục 7). Độ phủ chức năng đạt 100% ở mọi testbench mức khối.

Môi trường kiểm chứng đã tìm ra 12 lỗi hoặc giới hạn của thiết kế, trong đó có những lỗi nghiêm trọng như bộ kết nối bị treo sau một truy cập sai địa chỉ, mất dữ liệu ghi, đáp ứng đi tới sai master và hai lỗi làm sai dữ liệu ở các engine DMA.

Ngoài regression chính, toàn bộ test còn được chạy lại trên mã gốc nguyên bản của thư viện (283 lần chạy): mã gốc chỉ qua 163 lần, vì các lỗi nói trên làm hỏng phần lớn test của bộ kết nối và của axi_cdma (mục 7.2).

## 1.6. Bố cục báo cáo

Mục 2 giới thiệu bus AXI4. Mục 3 trình bày phương pháp kiểm chứng và VIP AXI4. Các mục 4, 5 và 6 lần lượt mô tả cách hoạt động và nội dung kiểm chứng của axi_ram, axi_crossbar, ba engine DMA cùng hệ thống tích hợp. Mục 7 tổng kết kết quả, kết quả chạy trên mã gốc, các lỗi tìm được, hạn chế và hướng phát triển; cuối báo cáo là danh mục tài liệu tham khảo.

# 2. GIỚI THIỆU BUS AXI4

## 2.1. Tổng quan

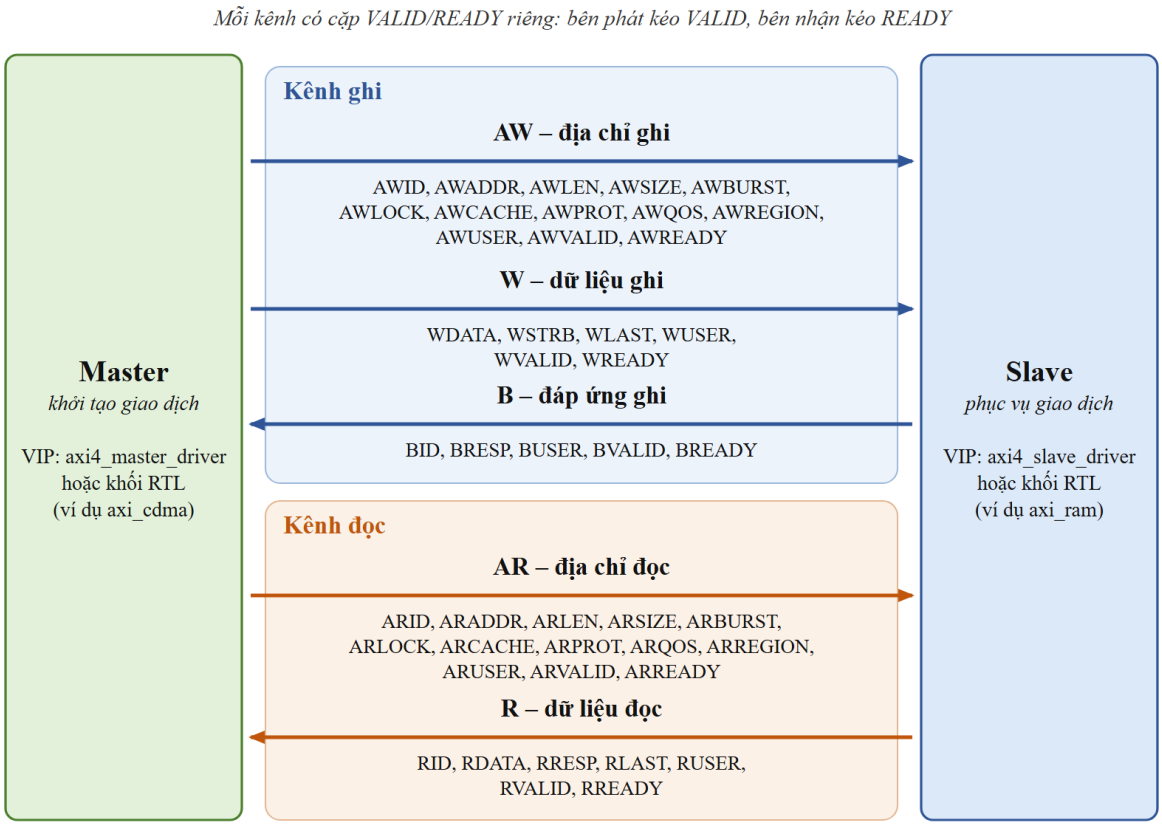
AXI (Advanced eXtensible Interface) thuộc họ giao thức AMBA của Arm [1]. Thế hệ AXI4 gồm ba biến thể: AXI4 (thường gọi là AXI4-Full) cho truy cập có địa chỉ với burst tới 256 nhịp; AXI4-Lite là bản rút gọn chỉ có giao dịch một nhịp, dùng cho thanh ghi điều khiển; AXI4-Stream cho luồng dữ liệu một chiều không có địa chỉ. Đồ án tập trung vào AXI4-Full; AXI4-Stream xuất hiện ở cổng dữ liệu của hai engine DMA.

Đặc điểm cốt lõi của AXI là tách giao dịch thành các kênh độc lập, mỗi kênh có cơ chế bắt tay riêng. Nhờ vậy địa chỉ, dữ liệu và đáp ứng có thể lệch pha nhau, nhiều giao dịch có thể cùng ở trạng thái đang chờ (outstanding), và có thể chèn thanh ghi vào từng kênh để đạt tần số cao.

## 2.2. Năm kênh và cơ chế bắt tay VALID/READY

*Bảng 2.1. Năm kênh của AXI4*

| Kênh | Hướng | Tín hiệu chính | Nội dung |
|---|---|---|---|
| AW – địa chỉ ghi | Master → Slave | AWID, AWADDR, AWLEN, AWSIZE, AWBURST, AWLOCK, AWCACHE, AWPROT, AWQOS, AWREGION, AWUSER, AWVALID, AWREADY | Địa chỉ và thuộc tính của một burst ghi |
| W – dữ liệu ghi | Master → Slave | WDATA, WSTRB, WLAST, WUSER, WVALID, WREADY | Các nhịp dữ liệu; WSTRB cho biết byte nào hợp lệ; WLAST đánh dấu nhịp cuối |
| B – đáp ứng ghi | Slave → Master | BID, BRESP, BUSER, BVALID, BREADY | Một đáp ứng cho cả burst ghi |
| AR – địa chỉ đọc | Master → Slave | ARID, ARADDR, ARLEN, ARSIZE, ARBURST, ARLOCK, ARCACHE, ARPROT, ARQOS, ARREGION, ARUSER, ARVALID, ARREADY | Địa chỉ và thuộc tính của một burst đọc |
| R – dữ liệu đọc | Slave → Master | RID, RDATA, RRESP, RLAST, RUSER, RVALID, RREADY | Các nhịp dữ liệu đọc, mỗi nhịp kèm đáp ứng; RLAST đánh dấu nhịp cuối |



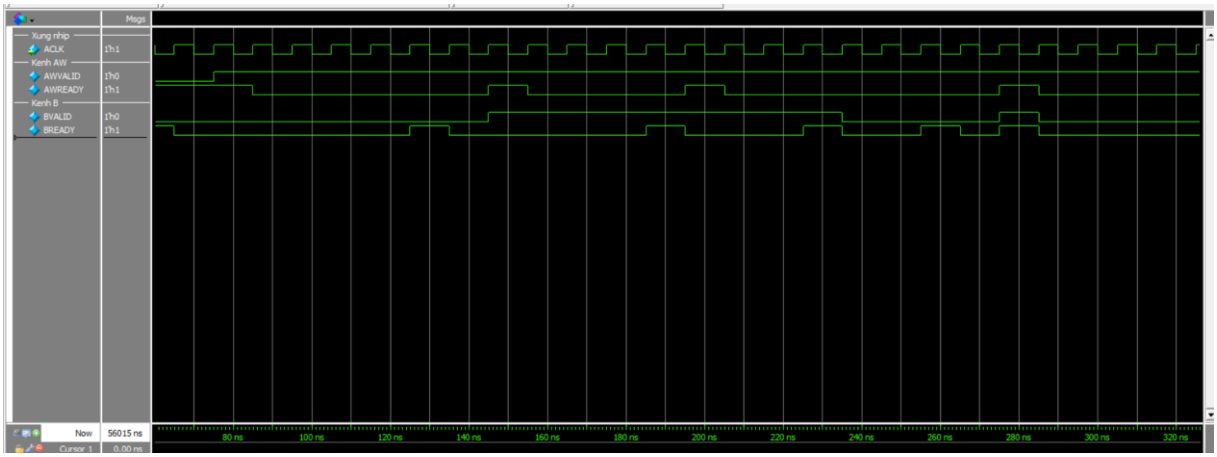

*Hình 2.1. Năm kênh AXI4 giữa master và slave*

Trên mỗi kênh, bên phát kéo VALID khi thông tin đã sẵn sàng, bên nhận kéo READY khi có thể nhận. Việc truyền xảy ra ở cạnh lên của xung nhịp mà tại đó cả VALID và READY cùng ở mức cao. Chuẩn đặt ra ba quy tắc quan trọng:

– Bên phát không được chờ READY rồi mới kéo VALID (tránh khoá chết).

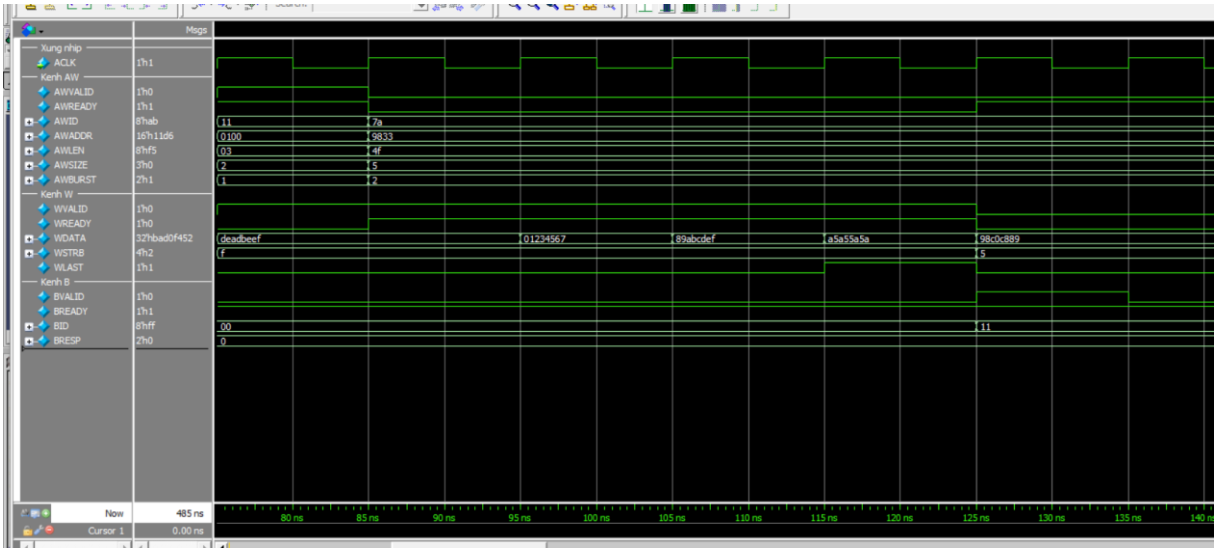
– Một khi VALID đã lên thì VALID và toàn bộ thông tin trên kênh phải giữ nguyên cho tới khi bắt tay xong.

– Bên nhận được phép chờ thấy VALID rồi mới kéo READY, hoặc kéo READY trước.



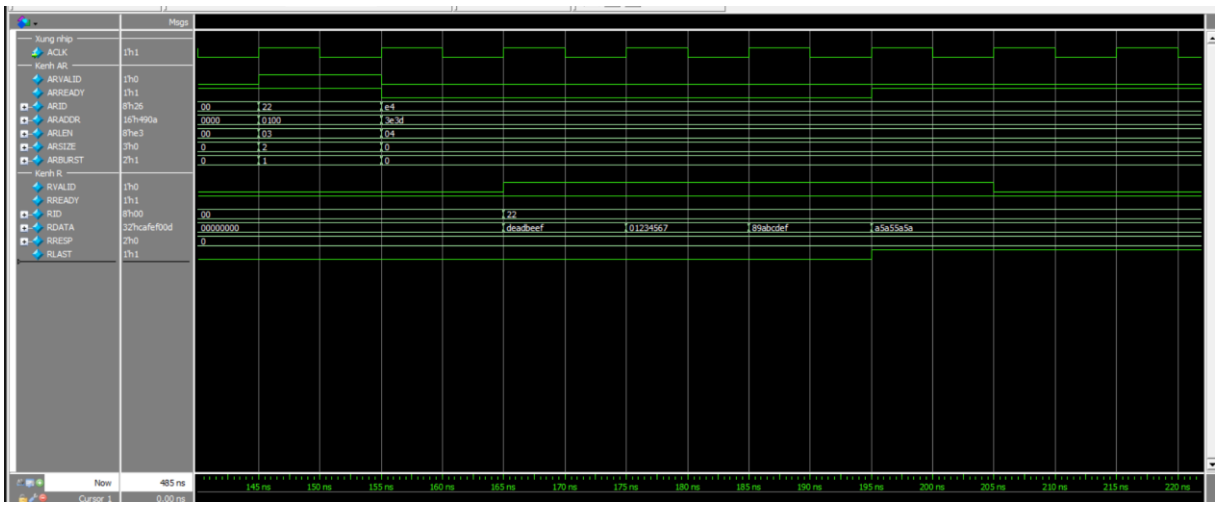

*Hình 2.2. Ba trường hợp bắt tay VALID/READY*

Hình 2.2 chụp từ mô phỏng axi_ram với bộ kích thích có điều khiển READY ngẫu nhiên. Trên kênh AW, bắt tay hoàn tất ở 85 ns khi READY đã ở mức cao trước khi VALID lên (trường hợp b), và ở 155 ns khi VALID đã lên từ trước và chờ READY (trường hợp a). Trên kênh B, VALID và READY cùng lên ở 285 ns (trường hợp c).




*Hình 2.3. Giao dịch ghi một burst INCR bốn nhịp*

Một giao dịch đọc gồm một lần truyền trên kênh AR và một hoặc nhiều nhịp trên kênh R; mỗi nhịp mang dữ liệu và mã đáp ứng riêng, nhịp cuối có RLAST = 1. Dữ liệu đọc chỉ được phát sau khi địa chỉ đã được nhận.




*Hình 2.4. Giao dịch đọc một burst bốn nhịp*

## 2.3. Burst, kích thước nhịp và strobe

- **AxLEN** là số nhịp trừ một. Burst INCR dài tối đa 256 nhịp; burst FIXED và WRAP dài tối đa 16 nhịp; burst WRAP chỉ được dài 2, 4, 8 hoặc 16 nhịp.
- **AxSIZE** là số byte của mỗi nhịp (2 mũ AxSIZE), không được lớn hơn độ rộng bus dữ liệu. Nhịp hẹp hơn bus gọi là narrow transfer; khi đó các byte lane hợp lệ dịch chuyển theo địa chỉ của từng nhịp.
- **AxBURST** chọn kiểu burst: FIXED (mọi nhịp cùng một địa chỉ), INCR (địa chỉ tăng dần theo AxSIZE), WRAP (tăng dần và quay vòng trong một khối có kích thước bằng số nhịp nhân với kích thước nhịp; địa chỉ bắt đầu phải thẳng hàng với AxSIZE).
- Một burst không được vượt qua biên 4 KB, vì các slave được ánh xạ theo trang tối thiểu 4 KB.
- Địa chỉ bắt đầu được phép lệch so với AxSIZE (unaligned); nhịp đầu khi đó chỉ mang các byte từ địa chỉ bắt đầu tới hết từ.
- **WSTRB** có một bit cho mỗi byte của WDATA; slave chỉ ghi những byte có bit strobe bằng 1.

## 2.4. ID, thứ tự và các giao dịch đang chờ

Mỗi giao dịch mang một ID. Các giao dịch cùng ID phải được trả lời theo đúng thứ tự phát; các giao dịch khác ID có thể được trả lời theo thứ tự bất kỳ. Trên kênh R, slave còn được phép xen kẽ các nhịp của những burst khác ID. AXI4 đã bỏ tín hiệu WID

của AXI3 nên dữ liệu ghi của các burst phải đi tuần tự, không xen kẽ. Một master có thể phát nhiều địa chỉ trước khi nhận xong đáp ứng của các giao dịch trước đó; số giao dịch đang chờ là một đại lượng quan trọng đối với bộ kết nối.

## 2.5. Mã đáp ứng và các tín hiệu phụ

*Bảng 2.2. Mã đáp ứng của AXI4*

| Mã | Tên | Ý nghĩa |
|---|---|---|
| 00 | OKAY | Truy cập thường thành công |
| 01 | EXOKAY | Truy cập độc quyền (exclusive) thành công |
| 10 | SLVERR | Slave nhận được truy cập nhưng báo lỗi |
| 11 | DECERR | Không có slave nào ở địa chỉ được truy cập; thường do bộ kết nối tự sinh ra |

Các tín hiệu phụ gồm AxLOCK (truy cập độc quyền), AxCACHE (thuộc tính bộ nhớ), AxPROT (bit 0: đặc quyền, bit 1: không an toàn, bit 2: lệnh), AxQOS (mức ưu tiên), AxREGION (vùng địa chỉ trong một slave) và các tín hiệu USER do người dùng định nghĩa. Bộ kết nối trong đồ án dùng AxPROT[1] để chặn truy cập không an toàn tới slave được đánh dấu là secure và tự tạo AxREGION từ kết quả giải mã địa chỉ.

## 2.6. AXI4-Stream

AXI4-Stream [2] chỉ có một kênh, cũng bắt tay bằng TVALID/TREADY. TDATA mang dữ liệu, TKEEP cho biết byte nào hợp lệ, TLAST đánh dấu nhịp cuối của một gói (frame), TID, TDEST và TUSER là thông tin kèm theo gói. axi_dma_rd xuất dữ liệu đọc từ bộ nhớ ra dưới dạng một gói AXI4-Stream; axi_dma_wr nhận một gói AXI4-Stream và ghi vào bộ nhớ.

## 2.7. Những yêu cầu của chuẩn được đồ án kiểm tra

- Tính ổn định của VALID và thông tin trên kênh cho tới khi bắt tay; không có giá trị X khi VALID ở mức cao.
- Tính hợp lệ của burst: độ dài WRAP, địa chỉ WRAP thẳng hàng, FIXED không quá 16 nhịp, INCR không vượt biên 4 KB.
- Số nhịp dữ liệu đúng với AxLEN; WLAST và RLAST đúng vị trí.
- BID và RID phải khớp với một giao dịch đang chờ; đáp ứng không được đến trước yêu cầu.

– Dữ liệu đúng từng byte theo strobe, kể cả với nhịp hẹp, địa chỉ lệch và burst WRAP.
– Hành vi khi reset: các VALID về mức thấp, giao dịch dở dang bị huỷ, thiết kế hoạt động lại bình thường.

# 3. PHƯƠNG PHÁP KIỂM CHỨNG VÀ AXI4 VIP

## 3.1. Phương pháp kiểm chứng

Đồ án áp dụng phương pháp kiểm chứng dựa trên độ phủ (coverage-driven verification) với UVM. Kích thích được sinh ngẫu nhiên trong giới hạn của các ràng buộc hợp lệ theo chuẩn; scoreboard so sánh hành vi của thiết kế với một mô hình tham chiếu; covergroup và độ phủ mã cho biết phần nào của không gian hành vi và phần nào của RTL đã được chạm tới. Năm nguyên tắc được giữ xuyên suốt:

- **Tự kiểm tra hoàn toàn.** Một lần chạy chỉ được tính là PASS khi log có dòng kết luận PASSED, không có UVM_ERROR hay UVM_FATAL, trình mô phỏng không báo lỗi, và số phép so sánh của scoreboard không nhỏ hơn ngưỡng tối thiểu của test. Test không so sánh gì thì bị coi là thất bại.
- **Ngẫu nhiên có ràng buộc, nhiều seed.** Mỗi test ngẫu nhiên chạy với 3 đến 12 seed; các góc khó được bổ sung bằng test có hướng viết ra sau khi đọc báo cáo độ phủ.
- **Nhiều cấu hình tham số.** Mỗi thiết kế được build với vài bộ tham số khác nhau, kết quả độ phủ được gộp lại.
- **Trung thực về độ phủ.** Chỉ các lần chạy PASS mới được gộp; chỉ loại trừ những mục chứng minh được bằng RTL là không thể đạt; báo cáo luôn đưa cả số trước và sau loại trừ.
- **Không che lỗi của thiết kế.** Mỗi lỗi tìm được có một test tái hiện riêng, đánh dấu XFAIL (bắt buộc thất bại). Nếu một test XFAIL bất ngờ PASS thì regression báo XPASS và coi là thất bại.

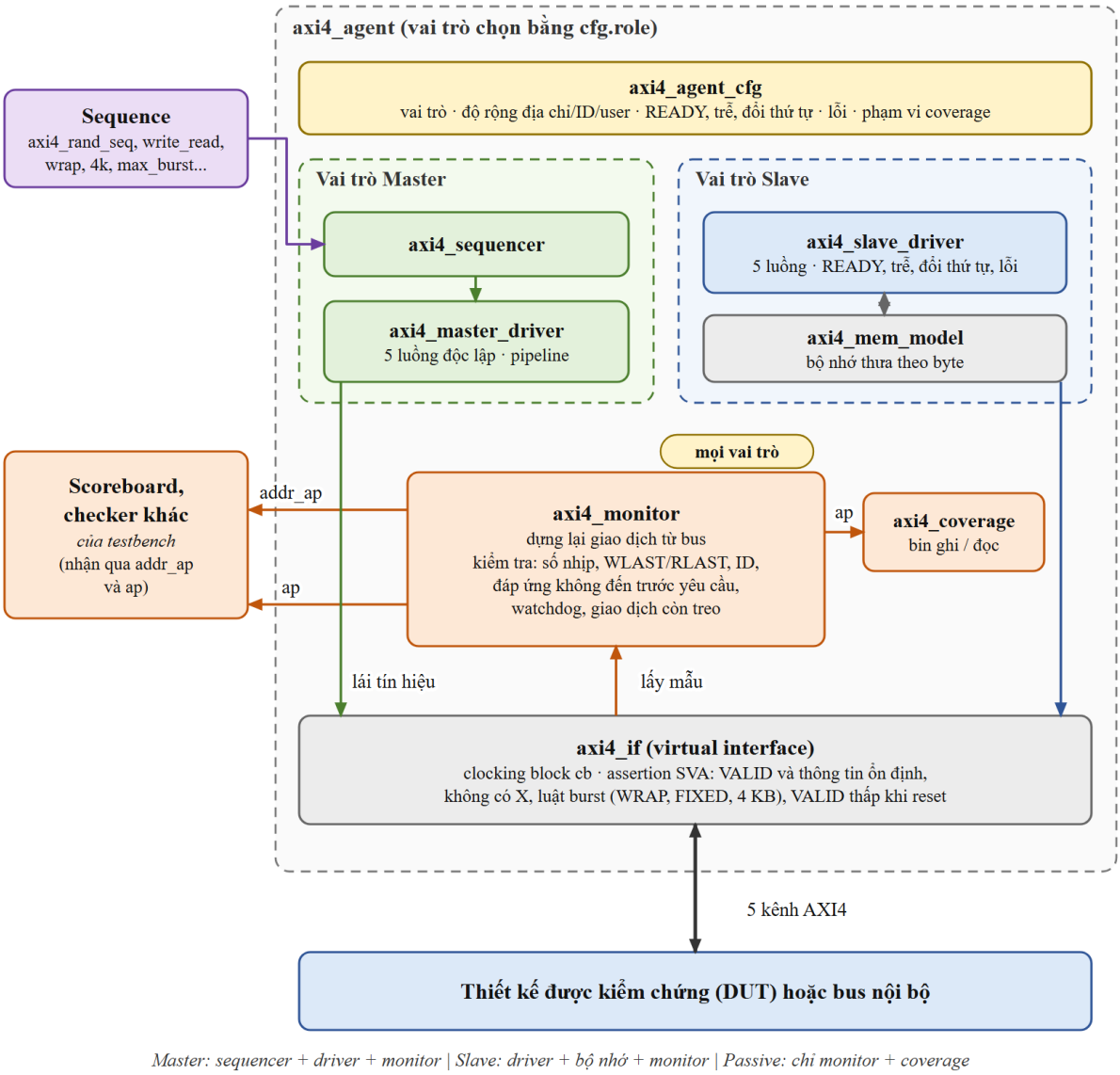
## 3.2. Kiến trúc của AXI4 VIP

VIP nằm trong thư mục axi4_vip và được đóng gói thành một package SystemVerilog duy nhất. Mọi cổng AXI trong mọi testbench của đồ án đều dùng cùng một bộ mã nguồn này, chỉ khác đối tượng cấu hình. Một agent có thể hoạt động ở một trong ba vai trò:

- **Master**: gồm sequencer, master driver và monitor; dùng để lái cổng slave của thiết kế.

– **Slave**: gồm slave driver phản ứng (reactive) có bộ nhớ riêng và monitor; dùng để đáp cổng master của thiết kế.

– **Passive**: chỉ có monitor, bộ kiểm tra giao thức và coverage; dùng để quan sát một bus nằm giữa hai khối RTL.




*Hình 3.1. Kiến trúc của AXI4 VIP*

*Bảng 3.1. Các thành phần của AXI4 VIP*

| Thành phần | Nhiệm vụ |
|---|---|
| axi4_if | Interface 5 kênh, clocking block và các assertion (SVA) kiểm tra giao thức ở mức tín hiệu |
| axi4_agent_cfg | Đối tượng cấu hình của một cổng: vai trò, độ rộng thật của địa chỉ/ID/user, mức backpressure, trễ, khả năng đổi thứ tự, phạm vi coverage |
| axi4_seq_item | Một burst AXI với các ràng buộc hợp lệ theo chuẩn |

| Thành phần | Nhiệm vụ |
|---|---|
| axi4_master_driver | Lái 5 kênh bằng 5 luồng độc lập, cho phép nhiều giao dịch đang chờ |
| axi4_slave_driver | Đáp ứng như một slave có bộ nhớ; điều khiển được backpressure, trễ, thứ tự đáp ứng, lỗi |
| axi4_monitor | Thu thập giao dịch từ bus và kiểm tra giao thức ở mức giao dịch |
| axi4_coverage | Độ phủ chức năng theo từng hướng ghi và đọc |
| axi4_mem_model, axi4_mem_scoreboard | Bộ nhớ thưa theo byte và scoreboard cho các thiết kế kiểu bộ nhớ |
| axi4_seq_lib | Thư viện sequence ngẫu nhiên và có hướng |

## 3.3. Hoạt động của các thành phần

***Transaction và ràng buộc***

Mỗi axi4_seq_item mô tả một burst: lệnh ghi hay đọc, ID, địa chỉ, độ dài, kích thước nhịp, kiểu burst, dữ liệu, strobe và các tín hiệu phụ. Các ràng buộc bảo đảm item sinh ra luôn hợp lệ: kích thước nhịp không vượt bus, độ dài và địa chỉ của WRAP đúng quy định, FIXED không quá 16 nhịp, INCR không vượt biên 4 KB. Strobe được sinh theo ba chế độ: đủ mọi byte hợp lệ, một phần, hoặc không byte nào.

***Master driver***

Driver nhận item từ sequencer và trả quyền điều khiển ngay, không chờ giao dịch kết thúc; sequence chờ cờ hoàn tất của từng item. Năm luồng chạy song song lái năm kênh, nên nhiều địa chỉ có thể được phát trước khi có đáp ứng, và dữ liệu ghi có thể đi trước hoặc sau địa chỉ. Cấu hình cho phép giới hạn số giao dịch đang chờ, chèn trễ ngẫu nhiên trước mỗi VALID và đặt xác suất BREADY, RREADY ở mức cao trong mỗi chu kỳ. Khi một kênh không có VALID, driver đặt giá trị ngẫu nhiên lên các tín hiệu của kênh đó để phát hiện thiết kế dùng thông tin mà không xét VALID.

***Slave driver***

Slave driver nhận địa chỉ và dữ liệu, thực hiện ghi và đọc trên một mô hình bộ nhớ thưa theo byte, rồi trả đáp ứng. Mỗi kênh là một luồng độc lập nên slave vẫn nhận địa chỉ mới trong khi đang trả dữ liệu. Có thể điều khiển xác suất của từng READY, trễ trước đáp ứng và giữa các nhịp đọc, cho phép trả lời không theo thứ tự giữa các ID, xen kẽ các nhịp đọc của những ID khác nhau, trả SLVERR hoặc DECERR cho toàn bộ, cho

một nhịp hoặc cho một vùng địa chỉ. Để kiểm chứng chính bộ kiểm tra của VIP, slave còn có thể cố ý phát RLAST sớm, phát BID sai hoặc không trả lời.

***Monitor và bộ kiểm tra giao thức***

Monitor lấy mẫu bus qua clocking block, dựng lại từng giao dịch và phát ra hai cổng phân tích: một cổng khi địa chỉ vừa được nhận, một cổng khi giao dịch hoàn tất. Trong lúc đó nó kiểm tra: số nhịp và vị trí WLAST, RLAST; BID và RID phải khớp một giao dịch đang chờ; đáp ứng không được đến trước yêu cầu; không có EXOKAY khi không phải truy cập độc quyền; không có X trên các byte hợp lệ; giao dịch không được treo quá một số chu kỳ; không còn giao dịch dở dang khi test kết thúc. Mỗi giao dịch hoàn tất được ghi một dòng vào log, gồm cả số chu kỳ bị chặn trên từng kênh.

***Assertion trong interface***

Interface chứa các assertion kiểm tra ở mức tín hiệu: VALID và thông tin trên kênh ổn định cho tới khi READY; không có X khi VALID ở mức cao; các quy tắc về burst; VALID ở mức thấp trong lúc reset. Vi phạm được báo qua cơ chế báo lỗi của UVM nên được tính vào kết luận của test.

***Độ phủ chức năng***

Coverage được thu riêng cho hướng ghi và hướng đọc của từng cổng: độ dài burst, kiểu burst, độ dài và điểm quay vòng của WRAP, kích thước nhịp, địa chỉ thẳng hàng hay lệch, mã đáp ứng, giá trị ID, số giao dịch đang chờ, các tín hiệu phụ, mức backpressure trên từng kênh và dạng strobe. Bin chỉ được tạo cho những hành vi mà cổng đó có thể thể hiện; ví dụ cổng của axi_ram không có bin đáp ứng lỗi vì RAM không bao giờ trả lỗi. Mỗi lần thu hẹp phạm vi như vậy đều có chú thích lý do trong mã testbench.

***Scoreboard bộ nhớ và thư viện sequence***

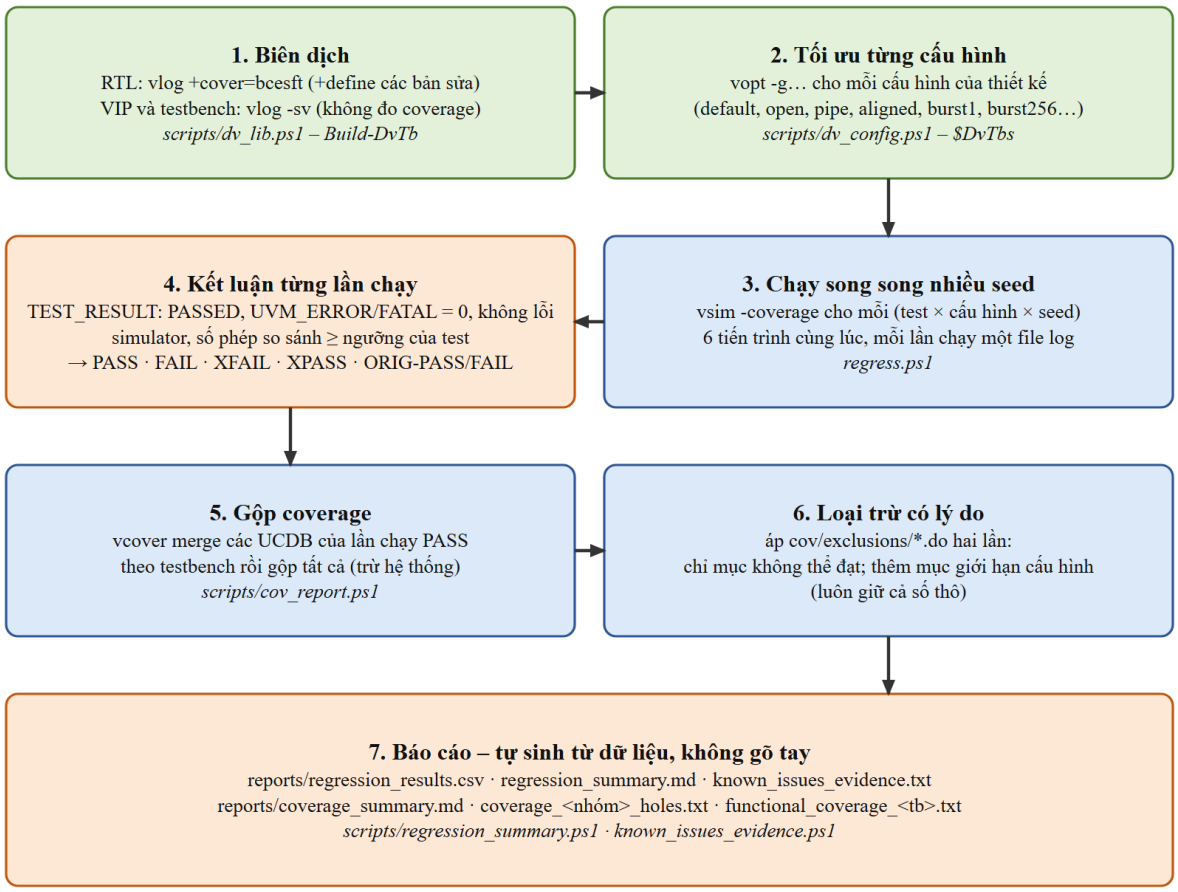
axi4_mem_scoreboard giữ một bộ nhớ tham chiếu theo byte. Mỗi lần ghi cập nhật bộ nhớ theo strobe; mỗi lần đọc được so từng byte với bộ nhớ tham chiếu. Scoreboard xử lý đúng trường hợp nhiều lần ghi chồng lên nhau khi đang chờ và trường hợp reset giữa chừng. Thư viện sequence gồm sequence ngẫu nhiên tổng quát có nhiều núm điều chỉnh, sequence ghi rồi đọc lại, sequence burst sát biên 4 KB, burst WRAP, burst 256 nhịp, quét vùng nhớ, và các sequence walking-1/walking-0 trên địa chỉ và dữ liệu.

## 3.4. Hạ tầng chung của các testbench

Mọi test kế thừa một lớp test cơ sở. Lớp này in phần đầu log gồm tên test, thiết kế và tham số, mục đích, kích thích, cách kiểm tra, seed và cấu hình của từng agent; cuối test nó in thống kê của scoreboard, lưu lượng trên bus, số phép so sánh và dòng kết luận. Ví dụ rút gọn của một log:

```
TEST      : dma_random_test
DUT       : axi_cdma (AXI_DATA_WIDTH=32 AXI_ADDR_WIDTH=32
AXI_MAX_BURST_LEN=16 ENABLE_UNALIGNED=1)
INTENT    : random transfers of every length class and alignment are correct under random
backpressure
STIMULUS  : 60 random descriptors: lengths around word/burst/4KB boundaries, random
unaligned addresses
CHECKING  : dma_scoreboard: per descriptor, bytes in order, byte count, tag, status length and
error code
SEED      : 1
[DMA_DESC] #1 accepted: descriptor rd_addr=57941031 wr_addr=a2a705e3 len=2 tag=e5
[AXI_TXN] mem: RD id=0 addr=57941031 len=0 size=2 INCR rresp=OKAY
stall(addr/data/resp)=1/0/0
scoreboard       : descriptors_checked=60 bytes_compared=... mismatches=0
checks performed : 21506 (minimum required 1)
TEST_RESULT: PASSED test=dma_random_test seed=1
```

Ba script điều khiển toàn bộ luồng: chạy một test với seed và tham số tuỳ chọn; chạy regression song song theo danh sách test, cấu hình build và số seed khai báo trong một file cấu hình; gộp cơ sở dữ liệu coverage của các lần chạy PASS, áp dụng các file loại trừ và sinh báo cáo. Mọi con số trong báo cáo này được lấy từ các file do những script đó sinh ra.




*Hình 3.2. Luồng chạy regression và tổng hợp độ phủ*

## 3.5. Tự kiểm chứng VIP

Trước khi dùng VIP để kiểm chứng thiết kế, bản thân VIP được kiểm chứng trong một testbench không có RTL: một agent master nối thẳng với một agent slave. Bốn test chức năng chứng minh driver, monitor, scoreboard và coverage hoạt động đúng; ba test âm chứng minh bộ kiểm tra giao thức thực sự bắt được lỗi khi slave cố ý vi phạm.

*Bảng 3.2. Các test tự kiểm chứng VIP*

| Test | Nội dung kiểm tra | Số lần chạy | Kết quả |
|---|---|---|---|
| vip_directed_test | Mọi sequence có hướng của thư viện chạy đúng trên bus luôn sẵn sàng | 2 | 2/2 PASS |
| vip_random_test | Lưu lượng ngẫu nhiên có pipeline dưới backpressure, đổi thứ tự và xen kẽ ngẫu nhiên | 10 | 10/10 PASS |
| vip_error_resp_test | Đáp ứng lỗi được trả, được truyền qua driver và monitor, và không làm hỏng bộ nhớ | 3 | 3/3 PASS |
| vip_reset_test | Reset giữa lưu lượng huỷ mọi giao dịch dở dang; hai agent hoạt động lại | 5 | 5/5 PASS |

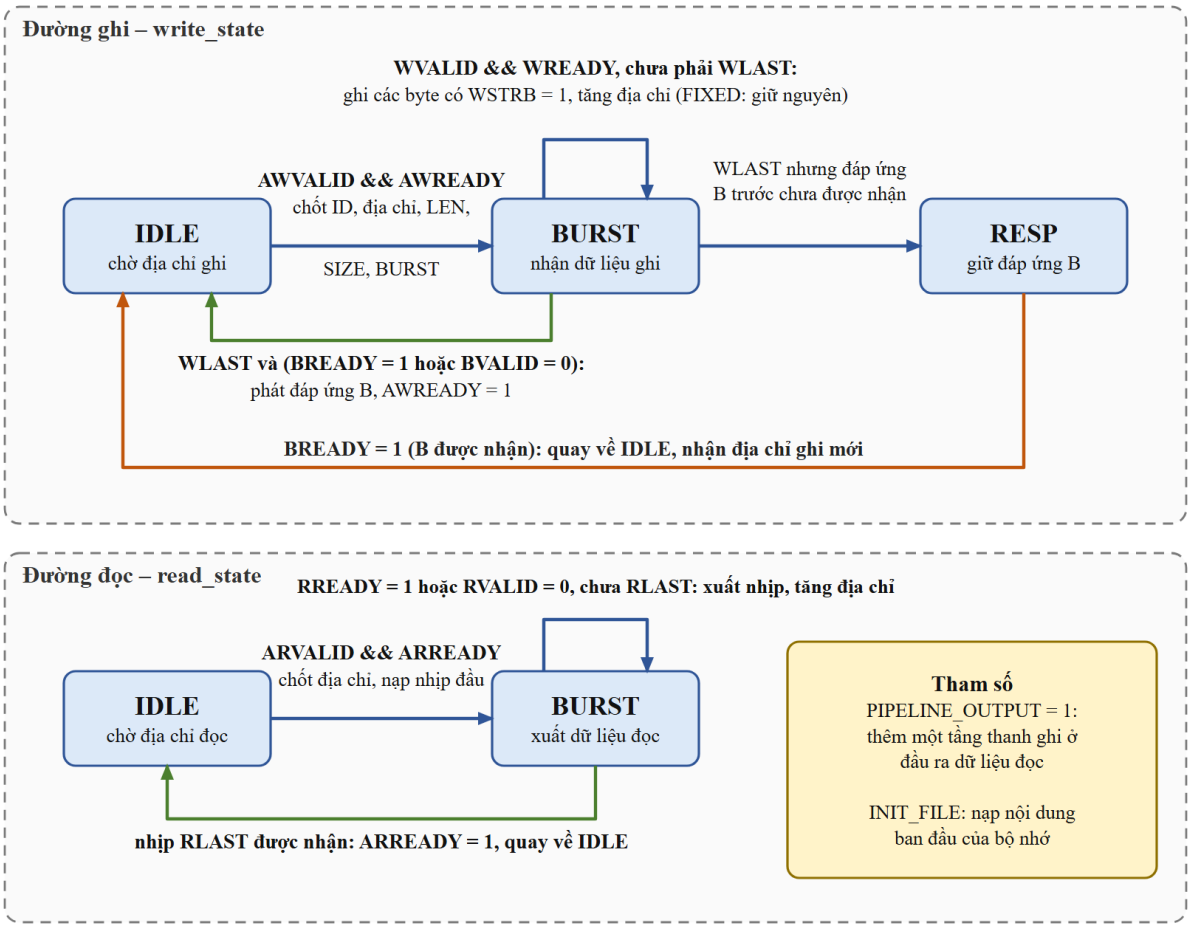
| Test | Nội dung kiểm tra | Số lần chạy | Kết quả |
|---|---|---|---|
| vip_neg_early_rlast_test | Test âm: bộ kiểm tra phát hiện RLAST phát sớm một nhịp | 1 | 1/1 PASS |
| vip_neg_wrong_bid_test | Test âm: bộ kiểm tra phát hiện BID không khớp giao dịch nào | 1 | 1/1 PASS |
| vip_neg_no_response_test | Test âm: bộ kiểm tra phát hiện giao dịch không bao giờ được trả lời | 1 | 1/1 PASS |

# 4. KIỂM CHỨNG BỘ NHỚ AXI_RAM

## 4.1. Cách hoạt động của axi_ram

axi_ram là một bộ nhớ có giao tiếp AXI4 slave. Trong đồ án nó được cấu hình với bus dữ liệu 32 bit, địa chỉ 16 bit (64 KB) và ID 8 bit. Đường ghi và đường đọc hoàn toàn độc lập, mỗi đường xử lý một burst tại một thời điểm.

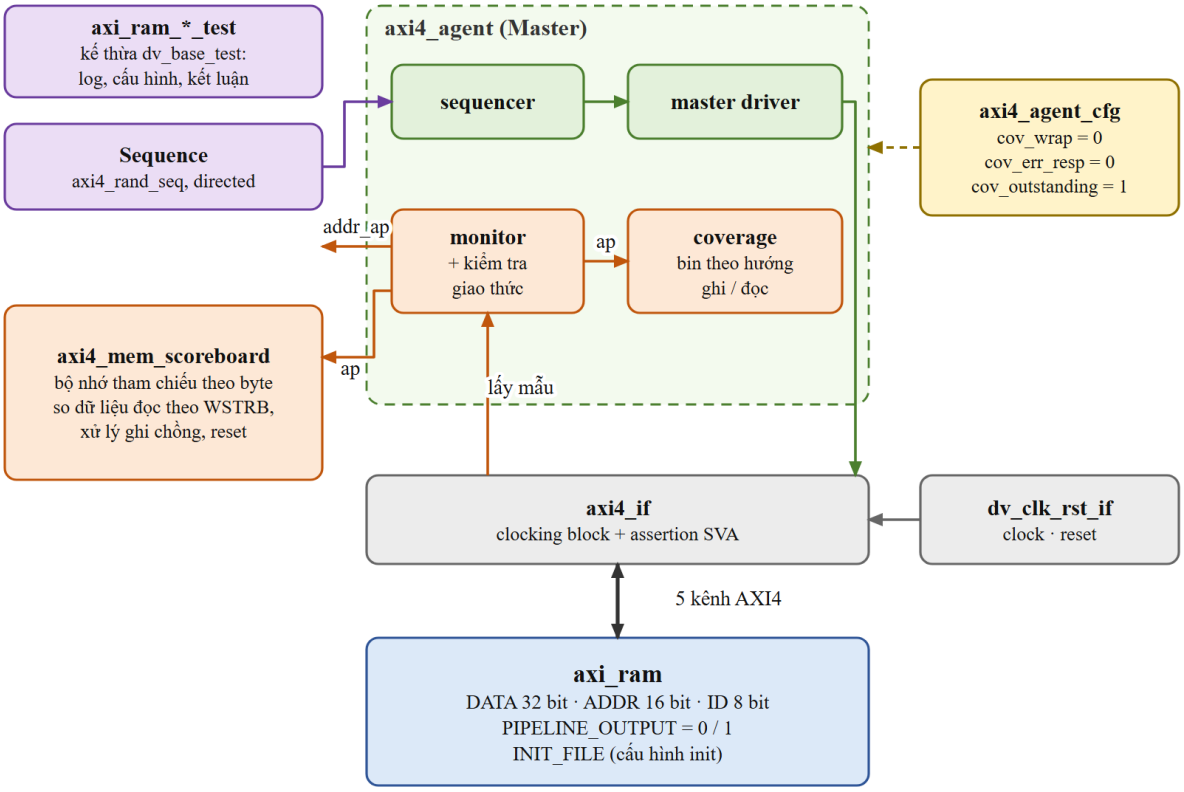
- **Đường ghi** là một máy trạng thái ba trạng thái. Ở trạng thái chờ, RAM nhận địa chỉ trên kênh AW và chốt ID, địa chỉ, độ dài, kích thước nhịp và kiểu burst. Ở trạng thái burst, RAM nhận từng nhịp trên kênh W, ghi các byte có strobe bằng 1 vào bộ nhớ và tăng địa chỉ theo kích thước nhịp (trừ khi burst là FIXED). Sau nhịp cuối RAM phát đáp ứng trên kênh B; nếu master chưa sẵn sàng, RAM chuyển sang trạng thái giữ đáp ứng cho tới khi BREADY lên.
- **Đường đọc** là một máy trạng thái hai trạng thái: nhận địa chỉ trên kênh AR, sau đó xuất lần lượt các nhịp dữ liệu trên kênh R và phát RLAST ở nhịp cuối.
- Tham số **PIPELINE_OUTPUT** chèn thêm một tầng thanh ghi ở đầu ra dữ liệu đọc để cải thiện timing.
- Tham số **INIT_FILE** (phần được thêm trong project) nạp nội dung ban đầu của bộ nhớ từ một file.
- RAM luôn trả đáp ứng OKAY và không có các cổng QOS, REGION, USER.




*Hình 4.1. Máy trạng thái ghi và đọc của axi_ram*

## 4.2. Testbench

Testbench gồm một agent master của VIP nối trực tiếp vào cổng của RAM và một axi4_mem_scoreboard. VIP được biên dịch đúng bằng độ rộng của RAM nên interface nối thẳng vào cổng. Cả hai cổng phân tích của monitor được nối vào scoreboard: cổng địa chỉ để xác định thứ tự của các lần ghi chồng nhau, cổng hoàn tất để cập nhật và so sánh dữ liệu. Ba cấu hình build được dùng: mặc định, PIPELINE_OUTPUT = 1, và có INIT_FILE.




*Hình 4.2. Testbench của axi_ram*

## 4.3. Các test đã thực hiện

*Bảng 4.1. Các test của axi_ram*

| Test | Nội dung kiểm tra | Cấu hình | Số lần chạy | Kết quả |
|---|---|---|---|---|
| axi_ram_smoke_test | Ghi rồi đọc lại một burst INCR bốn nhịp và một nhịp đơn ở cuối vùng nhớ | default, pipe | 2 | 2/2 PASS |
| axi_ram_directed_test | Walking-1/0 trên cả 16 bit địa chỉ và trên dữ liệu; burst INCR sát biên 4 KB; burst 256 nhịp; burst FIXED | default | 2 | 2/2 PASS |
| axi_ram_write_read_test | Tính toàn vẹn dữ liệu với mọi kiểu burst, kích thước nhịp, địa chỉ lệch và dạng strobe mà RAM hỗ trợ | default, pipe | 9 | 9/9 PASS |
| axi_ram_random_test | Ghi và đọc đồng thời, có pipeline, trên toàn bộ vùng nhớ, dưới backpressure ngẫu nhiên | default, pipe | 16 | 16/16 PASS |
| axi_ram_backpressure_test | RAM giữ đáp ứng và dữ liệu đọc khi master chưa sẵn sàng, kể cả khi burst kế tiếp đã được nhận | default, pipe | 7 | 7/7 PASS |
| axi_ram_reset_test | Reset giữa burst: RAM về trạng thái chờ, giữ nội dung đã ghi trước đó và hoạt động lại | default, pipe | 6 | 6/6 PASS |

| Test | Nội dung kiểm tra | Cấu hình | Số lần chạy | Kết quả |
|---|---|---|---|---|
| axi_ram_init_file_test | Nội dung nạp từ INIT_FILE đọc ra đúng và có thể bị ghi đè | init | 1 | 1/1 PASS |
| axi_ram_wrap_test | Burst WRAP phải quay vòng tại biên theo chuẩn AXI4 | default | 1 | XFAIL (RAM-01) |

## 4.4. Nhận xét

Mọi test chức năng đều PASS trên cả ba cấu hình. Test WRAP thất bại đúng như dự kiến: axi_ram chỉ phân biệt FIXED với các kiểu còn lại nên xử lý WRAP như INCR. Tài liệu của thư viện gốc ghi rõ axi_ram chỉ hỗ trợ FIXED và INCR, vì vậy đây là một giới hạn đã được công bố chứ không phải lỗi; test được giữ lại ở dạng XFAIL để ghi nhận hành vi. Trong các test ngẫu nhiên, burst WRAP chỉ được sinh ở dạng bắt đầu đúng biên quay vòng (không cần quay vòng) và bin độ phủ về quay vòng của cổng RAM được tắt.

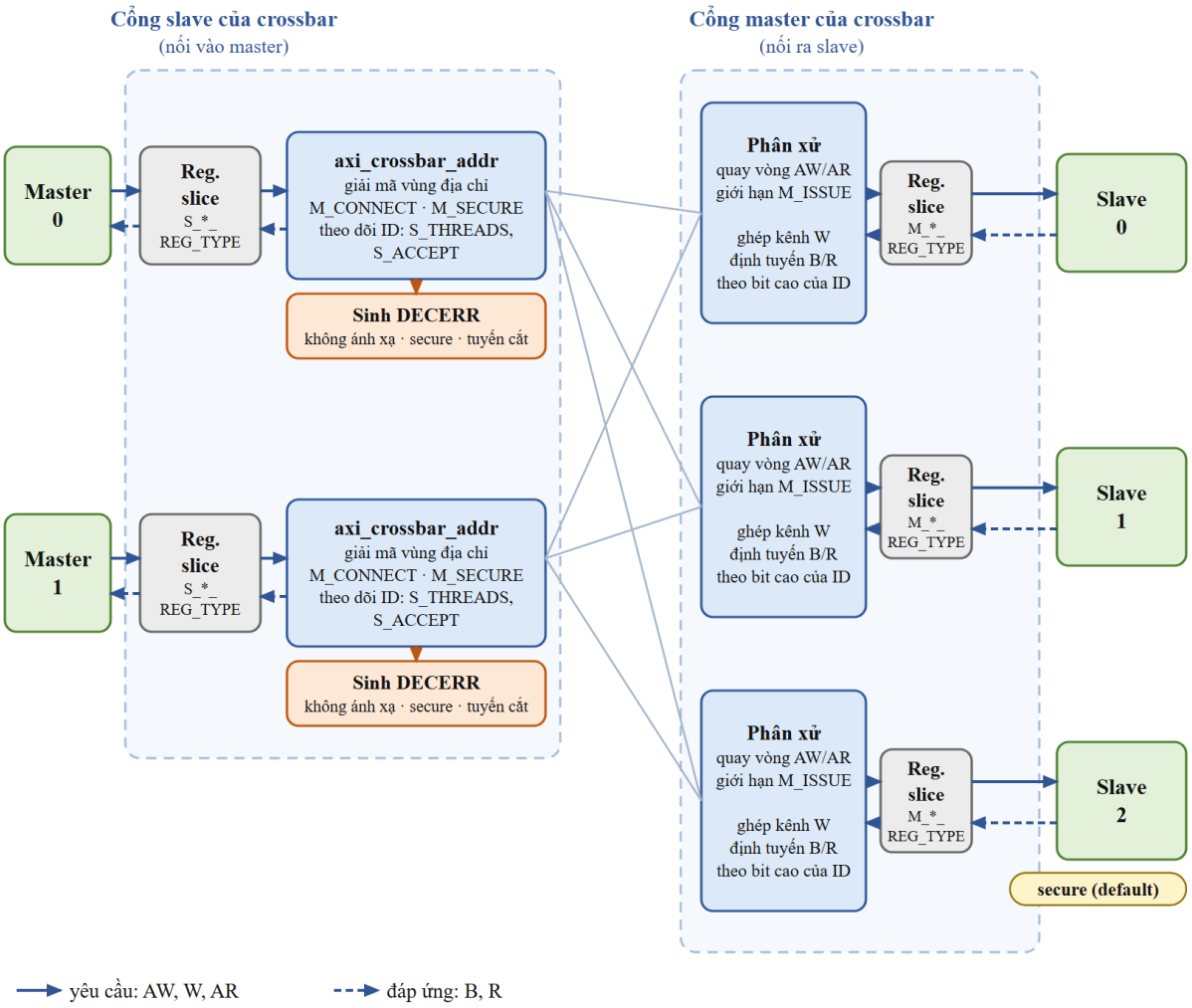
Độ phủ chức năng của cổng RAM đạt 100,00%. Độ phủ mã của axi_ram là 94,95% khi chưa loại trừ và 100,00% sau khi loại trừ 13 mục không thể đạt (các biến vòng lặp kiểu integer, một tín hiệu được khai báo nhưng không dùng, các đầu ra đáp ứng luôn bằng hằng, bit cao của kích thước nhịp bị RTL chặn ở giá trị tối đa, nhánh mặc định của các lệnh case đã liệt kê đủ trạng thái).

# 5. KIỂM CHỨNG BỘ KẾT NỐI AXI_CROSSBAR

## 5.1. Cách hoạt động của axi_crossbar

axi_crossbar nối S_COUNT cổng slave (nơi các master cắm vào) với M_COUNT cổng master (nơi các slave cắm vào). Đường ghi (axi_crossbar_wr) và đường đọc (axi_crossbar_rd) tách biệt hoàn toàn, nên một giao dịch ghi không bao giờ chặn một giao dịch đọc. Các khối chức năng chính:

- **Giải mã địa chỉ (axi_crossbar_addr)**, mỗi cổng slave có một khối cho mỗi hướng. Địa chỉ được so với bảng các vùng địa chỉ của từng slave (mỗi slave có thể có nhiều vùng). Kết quả là slave đích và chỉ số vùng, chỉ số này được đưa ra tín hiệu AxREGION. Nếu không khớp vùng nào, hoặc tuyến từ master tới slave không được nối theo tham số M_CONNECT, hoặc slave được đánh dấu secure mà truy cập có AxPROT[1] = 1, bộ kết nối tự trả đáp ứng DECERR.
- **Bảo toàn thứ tự theo ID.** Khối giải mã theo dõi các ID đang có giao dịch chờ. Một giao dịch mới có cùng ID với giao dịch đang chờ chỉ được chuyển đi nếu nó tới cùng slave và cùng vùng; nếu không nó bị giữ lại, nhờ đó các đáp ứng cùng ID không thể đảo thứ tự. Số ID theo dõi đồng thời giới hạn bởi S_THREADS; số giao dịch đang chờ của một master giới hạn bởi S_ACCEPT.
- **Phân xử.** Mỗi cổng master có một bộ phân xử quay vòng (round-robin) chọn master được phát địa chỉ tới slave đó; số giao dịch đang chờ tại một slave giới hạn bởi M_ISSUE. Dữ liệu ghi được định tuyến theo thứ tự các địa chỉ đã được cấp quyền.
- **Định tuyến đáp ứng.** ID gửi tới slave được nối thêm chỉ số của master ở các bit cao; khi đáp ứng quay về, các bit này cho biết phải trả cho master nào. Mỗi cổng slave có một bộ phân xử chọn giữa đáp ứng của các slave và đáp ứng DECERR tự sinh.
- **Register slice (axi_register_wr, axi_register_rd).** Mỗi kênh của mỗi cổng có thể chèn một thanh ghi kiểu bỏ qua (0), bộ đệm đơn (1) hoặc bộ đệm skid (2) để cắt đường timing.




Đường ghi (axi_crossbar_wr) và đường đọc (axi_crossbar_rd) có cùng cấu trúc và tách biệt hoàn toàn; hình vẽ chung cho cả hai. Đường xám: mọi master có thể tới mọi slave được nối.

*Hình 5.1. Sơ đồ khối của axi_crossbar*

## 5.2. Cấu hình được kiểm chứng

Bộ kết nối được cấu hình với 2 master và 3 slave, địa chỉ và dữ liệu 32 bit, ID phía master 4 bit (phía slave 5 bit), tín hiệu USER 2 bit. Bản đồ địa chỉ được chọn sao cho mọi bit địa chỉ đều có thể thay đổi trên mọi cổng slave, có vùng nhỏ nhất (4 KB), có vùng không dùng và có nhiều khoảng trống không ánh xạ.

*Bảng 5.1. Bản đồ địa chỉ của cấu hình kiểm chứng*

| Slave | Vùng 0 | Vùng 1 | Vùng 2 | Ghi chú |
|---|---|---|---|---|
| 0 | 0x0000_0000, 256 MB | 0xF000_0000, 256 MB | 0x1000_0000, 4 KB | |
| 1 | 0x3000_0000, 256 MB | 0xC000_0000, 256 MB | không dùng | Master 1 không được ghi (build default) |
| 2 | 0x5000_0000, 256 MB | 0xA000_0000, 256 MB | 0x8000_0000, 64 KB | Secure; master 0 không được đọc (build default) |

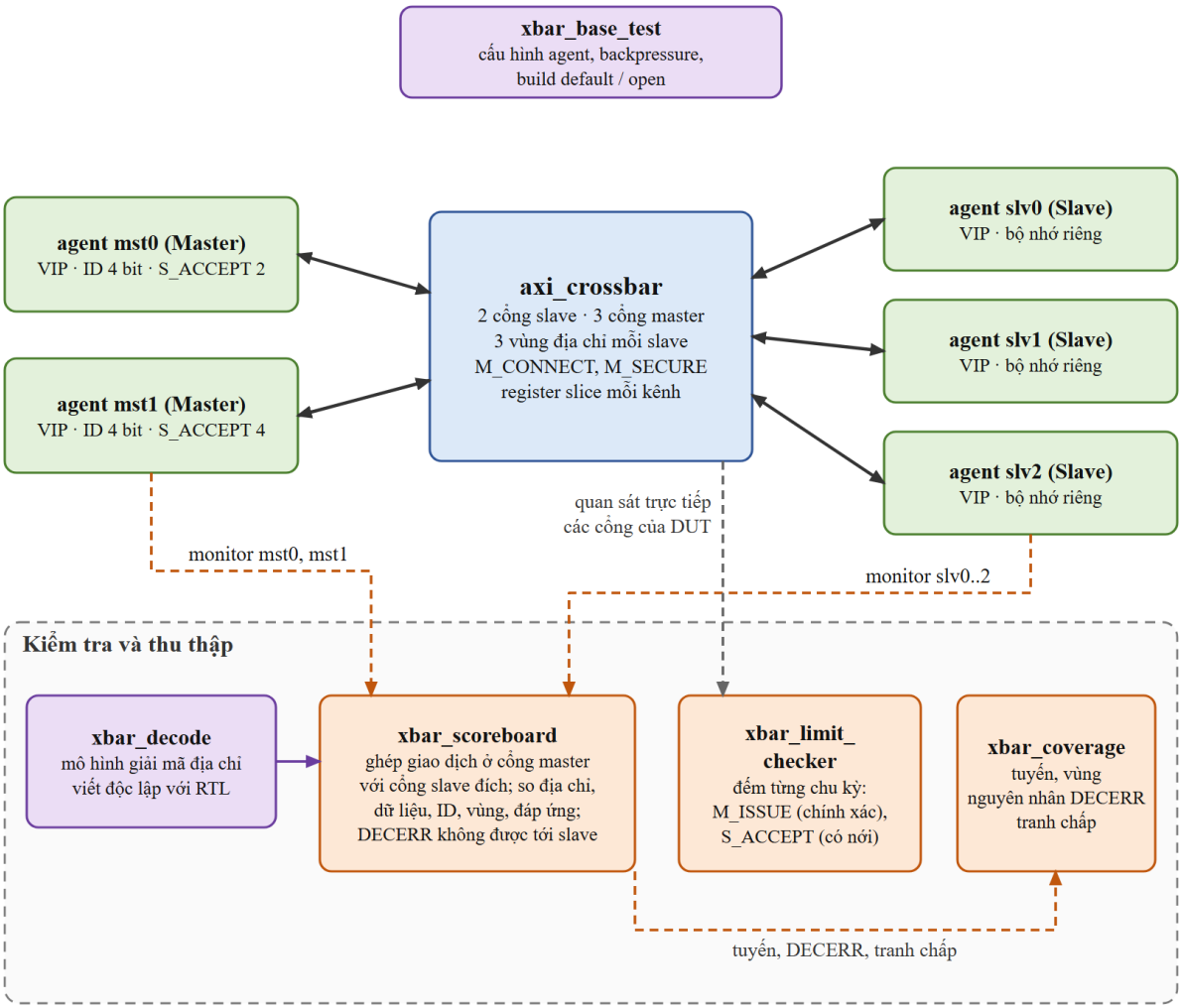
Các giới hạn đồng thời được đặt khác nhau cho từng cổng để mọi giới hạn đều bị chạm tới: S_THREADS là 4 và 2, S_ACCEPT là 2 và 4 cho master 0 và 1. Trên mỗi kênh, cả ba kiểu register slice đều xuất hiện ở ít nhất một cổng. Hai cấu hình build được dùng: **default** có tuyến bị cắt, có slave secure và M_ISSUE bằng 4, 2, 1 cho slave 0, 1, 2; **open** nối mọi tuyến, không có slave secure và hoán vị M_ISSUE thành 1, 4, 2 để mỗi cổng slave đều được kiểm chứng với cả giới hạn bằng một lẫn giới hạn lớn hơn.

## 5.3. Testbench

Hai agent master của VIP đóng vai hai master, ba agent slave của VIP đóng vai ba slave. Ngoài các kiểm tra giao thức của VIP trên cả năm cổng, testbench có ba thành phần riêng:

- **Mô hình tham chiếu của bộ giải mã địa chỉ**, viết độc lập với RTL từ mô tả chức năng: với mỗi truy cập, nó cho biết slave và vùng đích, hoặc lý do phải trả DECERR (không ánh xạ, secure, tuyến không nối).
- **Scoreboard định tuyến**: mỗi giao dịch thấy ở cổng master được ghép với giao dịch tương ứng ở cổng slave mà mô hình tham chiếu chỉ ra, rồi so sánh địa chỉ, độ dài, kiểu burst, dữ liệu, strobe, ID (kể cả phần chỉ số master), vùng và đáp ứng. Truy cập phải bị từ chối thì phải nhận DECERR và không được xuất hiện ở bất kỳ slave nào.
- **Bộ kiểm tra giới hạn đồng thời**: đếm từng chu kỳ số giao dịch đang chờ ở mỗi cổng và so với M_ISSUE (chính xác ở cổng slave) và S_ACCEPT (có nới theo độ sâu của register slice ở cổng master).

Độ phủ chức năng riêng của bộ kết nối gồm: tuyến (master, hướng, slave) và vùng của từng slave; nguyên nhân DECERR theo hướng và theo master; tình huống một master hay cả hai master cùng truy cập một slave.




*Hình 5.2. Testbench của axi_crossbar*

## 5.4. Các test đã thực hiện

*Bảng 5.2. Các test của axi_crossbar (build chính, đã bật các bản sửa tuỳ chọn)*

| Test | Nội dung kiểm tra | Cấu hình | Số lần chạy | Kết quả |
|---|---|---|---|---|
| xbar_smoke_test | Mỗi master tới được mọi vùng của mọi slave mà nó được nối, lần lượt từng truy cập | default, open | 2 | 2/2 PASS |
| xbar_param_test | Kiểu register slice được dựng trong thiết kế đúng với tham số yêu cầu | default, open | 2 | 2/2 PASS |
| xbar_directed_test | Walking-1/0 trên 32 bit địa chỉ và trên dữ liệu, burst 256 nhịp, WRAP có quay vòng, burst sát biên 4 KB, ID toàn 0 và toàn 1 qua bộ kết nối | default, open | 3 | 3/3 PASS |
| xbar_random_test | Lưu lượng ngẫu nhiên đồng thời của hai master tới mọi vùng và cả vùng không ánh xạ, dưới backpressure ngẫu nhiên | default, open | 18 | 18/18 PASS |

| Test | Nội dung kiểm tra | Cấu hình | Số lần chạy | Kết quả |
|---|---|---|---|---|
| xbar_decerr_test | Địa chỉ không ánh xạ, truy cập không an toàn tới slave secure và tuyến không nối đều nhận DECERR và không tới slave nào | default | 4 | 4/4 PASS |
| xbar_id_thread_test | Các giao dịch cùng ID tới nhiều slave giữ đúng thứ tự; các giới hạn S_THREADS, S_ACCEPT, M_ISSUE chặn mà không làm mất giao dịch | default, open | 9 | 9/9 PASS |
| xbar_issue_limit_test | Hai master dồn lưu lượng vào từng slave trả lời chậm để chạm và giải phóng giới hạn M_ISSUE, S_ACCEPT | default, open | 6 | 6/6 PASS |
| xbar_backpressure_test | Mọi register slice và bộ ghép kênh giữ nguyên dữ liệu khi phía sau chưa sẵn sàng | default, open | 6 | 6/6 PASS |
| xbar_slave_error_test | Đáp ứng lỗi do slave phát ra về đúng master, không bị thay đổi | default, open | 6 | 6/6 PASS |
| xbar_slave_skew_test | Địa chỉ và dữ liệu ghi tới slave lệch nhau theo cả hai chiều với khoảng cách bất kỳ | default, open | 5 | 5/5 PASS |
| xbar_decerr_backlog_test | Các đáp ứng DECERR xếp hàng đúng khi master nhận chậm | default | 3 | 3/3 PASS |
| xbar_decerr_lockup_test | Sau các đáp ứng DECERR, master vẫn dùng được mọi slave | default | 2 | 2/2 PASS |
| xbar_decerr_wdata_test | Dữ liệu của lần ghi hợp lệ đi ngay sau một lần ghi bị DECERR tới slave đầy đủ | default | 3 | 3/3 PASS |
| xbar_reset_test | Reset giữa lưu lượng đưa bộ kết nối về trạng thái chờ; sau đó định tuyến đúng | default | 4 | 4/4 PASS |
| xbar_reset_drop_test | Reset đúng lúc đang bỏ dữ liệu của một lần ghi bị DECERR, sau đó ghi với dữ liệu đi trước địa chỉ | default | 3 | 3/3 PASS |
| xbar_decerr_b_order_test | Đáp ứng DECERR của lần ghi chỉ được phát sau khi dữ liệu ghi đã được nhận | default | 1 | XFAIL (XBAR-01) |
| xbar_decerr_id_order_test | Đáp ứng DECERR giữ đúng vị trí giữa các đáp ứng cùng ID | default | 1 | XFAIL (XBAR-03) |
| xbar_read_interleave_test | Dữ liệu đọc mà slave xen kẽ giữa các ID chỉ tới đúng master | default | 1 | XFAIL (XBAR-04) |

## 5.5. Các lỗi tìm được ở bộ kết nối

Tám vấn đề được phát hiện ở bộ kết nối, ký hiệu XBAR-01 đến XBAR-08. Vì tám file RTL của bộ kết nối giống hệt mã gốc nên đây đều là hành vi của thư viện gốc. Ba trong số đó (XBAR-01, XBAR-02, XBAR-06) đã được người dùng khác báo trên kho mã gốc [7]; XBAR-04 là tính năng chưa hỗ trợ mà chính tác giả đã ghi nhận [7].

- **XBAR-01** – Đáp ứng DECERR được phát trước khi yêu cầu được nhận xong: với lần ghi sai địa chỉ, BVALID lên trước nhịp dữ liệu cuối, trái với quy định đáp ứng ghi phải theo sau nhịp ghi cuối. Các test khác miễn trừ thông báo này nhưng có đếm số lần xảy ra.
- **XBAR-02** – Bị treo sau DECERR: giao dịch bị từ chối không được đếm lúc bắt đầu nhưng vẫn bị trừ lúc hoàn tất, bộ đếm giao dịch đang chờ của master bị trừ xuống dưới 0 và quay vòng thành giá trị cực đại, từ đó master bị chặn vĩnh viễn.
- **XBAR-03** – DECERR không giữ thứ tự với các đáp ứng cùng ID: nhánh sinh DECERR không qua cơ chế theo dõi ID nên đáp ứng DECERR có thể vượt lên trước đáp ứng của một giao dịch hợp lệ phát trước đó.
- **XBAR-04** – Khi slave xen kẽ các nhịp đọc của hai burst thuộc hai master, một master nhận được nhịp không phải của mình. Tác giả thư viện đã ghi nhận việc chưa hỗ trợ xen kẽ dữ liệu đọc, nên đây là tính năng chưa hỗ trợ.
- **XBAR-05** – Mất nhịp ghi đầu tiên sau một lần ghi bị DECERR: tín hiệu WREADY trả về master không được chặn khi chưa có lệnh ghi hợp lệ, nên nhịp dữ liệu được nhận nhưng không được chuyển tới đâu.
- **XBAR-06** – Các tham số chọn kiểu register slice phía cổng master không được truyền từ axi_crossbar xuống hai khối con, nên luôn mang giá trị mặc định.
- **XBAR-07** – Khi kênh đáp ứng phía cổng master có thanh ghi, master nhận đáp ứng được chọn bằng ID ở phía trước thanh ghi trong khi đáp ứng lấy ở phía sau, dẫn tới trả lời nhầm master. Lỗi này bị XBAR-06 che đi và chỉ lộ ra sau khi XBAR-06 được sửa.
- **XBAR-08** – Giới hạn M_ISSUE bị vượt khi hai master cùng xin đọc một slave; với M_ISSUE bằng 1 bộ đếm một bit bị tràn và giới hạn mất tác dụng hoàn toàn.

Hai lỗi XBAR-06, XBAR-07 và lỗi XBAR-08 được tìm ra nhờ phân tích những lỗ độ phủ không giải thích được: một hàng điều kiện không bao giờ đạt dù testbench tạo đúng tình huống ở cổng, và số giao dịch đang chờ ở một slave lớn hơn giới hạn được cấu hình. Năm lỗi XBAR-02, 05, 06, 07, 08 có bản sửa tuỳ chọn đặt trong các khối biên dịch có điều kiện; khi không định nghĩa macro thì RTL giữ nguyên hành vi gốc. Regression chính và số liệu độ phủ được đo trên RTL đã bật các bản sửa này; RTL gốc và các bản chỉ sửa một phần được build riêng để chạy test minh hoạ từng lỗi.

# 6. KIỂM CHỨNG CÁC ENGINE DMA VÀ HỆ THỐNG TÍCH HỢP

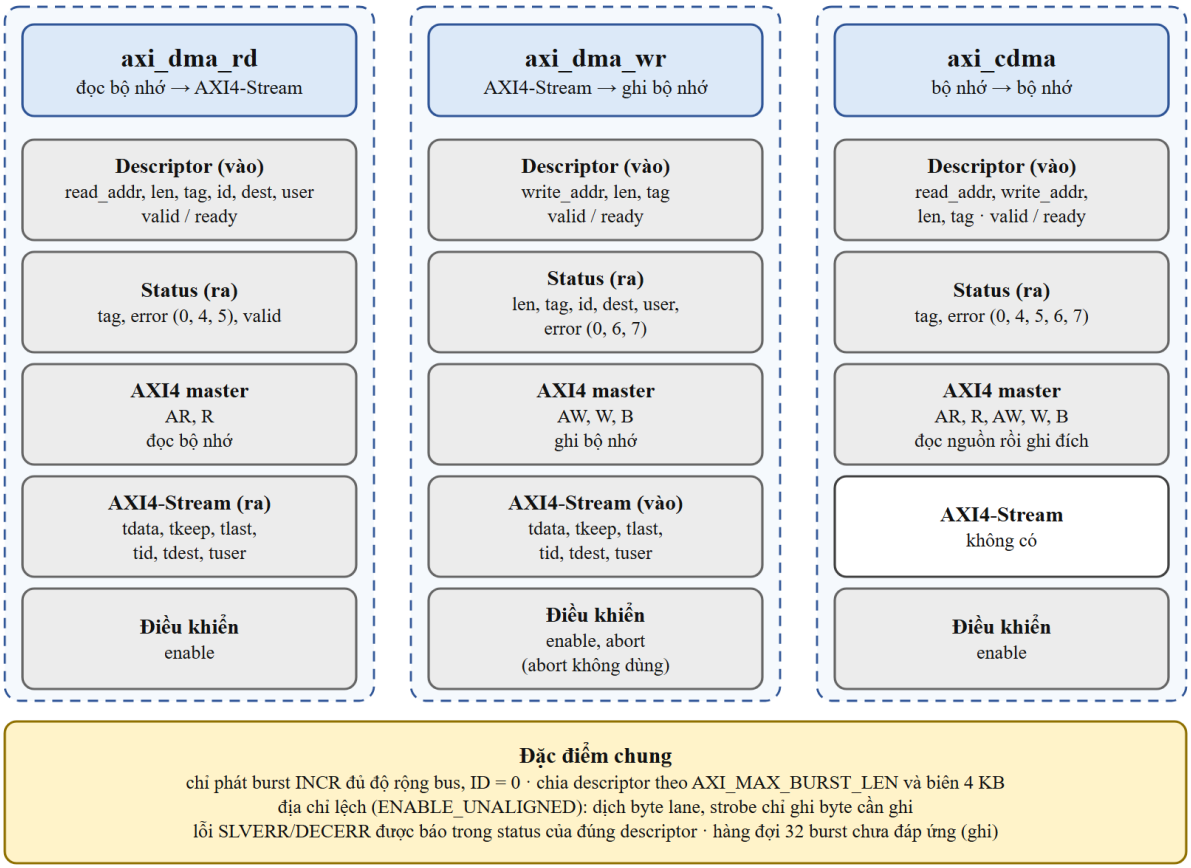
## 6.1. Cách hoạt động của ba engine DMA

Cả ba engine đều nhận lệnh dưới dạng descriptor qua một cổng có bắt tay valid/ready, thực hiện việc truyền trên một cổng AXI4 master, rồi báo kết quả qua một cổng status. Mỗi descriptor có một nhãn (tag) được trả lại trong status để phần mềm nhận biết lệnh nào đã xong.

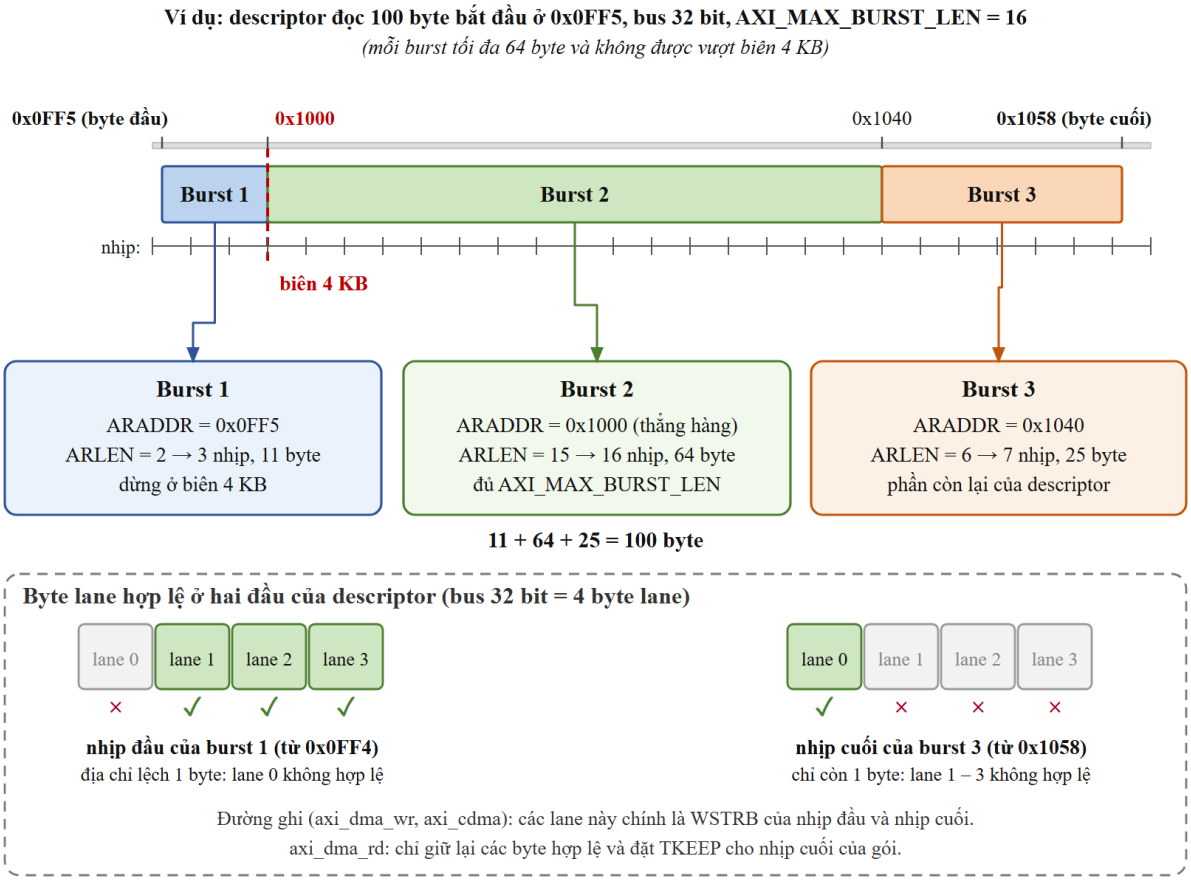
- **axi_dma_rd**: descriptor gồm địa chỉ, độ dài (tối đa 2^20 − 1 byte), tag và các trường id, dest, user của gói. Engine đọc đúng số byte đó từ bộ nhớ qua AXI và xuất ra một gói AXI4-Stream; TKEEP của nhịp cuối cho biết số byte hợp lệ.
- **axi_dma_wr**: descriptor gồm địa chỉ, độ dài tối đa và tag. Engine nhận một gói AXI4-Stream và ghi vào bộ nhớ. Nếu gói ngắn hơn descriptor, engine dừng sớm và báo số byte thực ghi; nếu gói dài hơn, phần dư bị bỏ. Status mang cả id, dest, user của gói.
- **axi_cdma**: descriptor gồm địa chỉ nguồn, địa chỉ đích, độ dài và tag. Engine đọc từ nguồn và ghi sang đích; hai địa chỉ có thể lệch byte khác nhau.

Các đặc điểm chung về cách phát burst:

- Engine chỉ phát burst INCR đủ độ rộng bus, ID bằng 0.
- Một descriptor được chia thành nhiều burst sao cho mỗi burst không dài quá AXI_MAX_BURST_LEN nhịp và không vượt biên 4 KB.
- Với ENABLE_UNALIGNED = 1, địa chỉ có thể bắt đầu ở byte bất kỳ; engine dịch dữ liệu giữa các byte lane và dùng strobe để chỉ ghi đúng các byte cần ghi.
- Lỗi SLVERR hoặc DECERR từ bộ nhớ được ghi nhận và báo trong status của đúng descriptor bằng mã lỗi riêng cho đọc và cho ghi; engine vẫn tiếp tục hoạt động.
- Engine ghi có một hàng đợi 32 phần tử cho các burst chưa nhận được đáp ứng; khi hàng đợi đầy, engine phải ngừng phát burst mới.
- Ngõ vào enable ở mức thấp ngăn engine nhận descriptor mới, nhưng không ảnh hưởng tới việc truyền đang diễn ra.




*Hình 6.1. Sơ đồ khối và các cổng của ba engine DMA*




*Hình 6.2. Chia một descriptor thành các burst AXI*

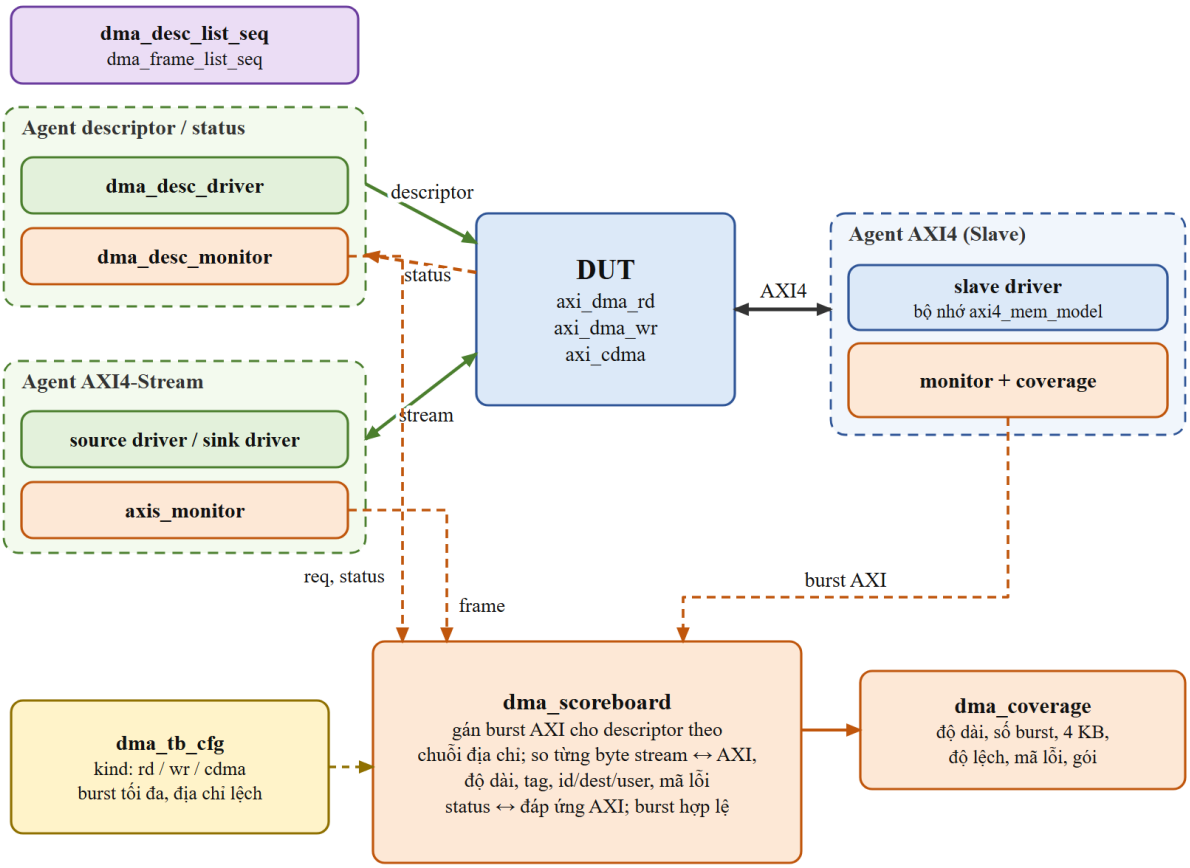
## 6.2. Testbench

Ba engine dùng chung một package testbench, mỗi engine có một module top riêng. Cổng AXI của engine nối với một agent slave của VIP đóng vai bộ nhớ. Testbench có thêm một agent cho cổng descriptor và status, và một agent AXI4-Stream (phát gói cho axi_dma_wr, nhận gói từ axi_dma_rd). Agent AXI4-Stream phát cả giá trị ngẫu nhiên lên TLAST và dữ liệu khi TVALID ở mức thấp để kiểm tra engine chỉ dùng chúng khi có TVALID.

Scoreboard DMA làm việc theo từng descriptor. Nó gán mỗi burst AXI quan sát được cho descriptor tương ứng bằng cách lần theo chuỗi địa chỉ (burst đầu bắt đầu tại địa chỉ của descriptor, burst sau bắt đầu ở chỗ burst trước kết thúc), thu thập các byte đã đọc và đã ghi trên AXI, rồi khi status về thì kiểm tra:

- Từng byte của gói so với byte tương ứng trên AXI, đúng thứ tự và đủ số lượng; với axi_cdma là byte ghi ra đích so với byte đọc từ nguồn.
- Không byte nào bị ghi hai lần, không byte nào ngoài phạm vi của descriptor bị ghi.
- Tag, độ dài trong status, các trường id, dest, user.
- Mã lỗi trong status khớp với đáp ứng lỗi đã thấy trên AXI.
- Tính hợp lệ của burst: đủ độ rộng, kiểu INCR, không dài quá giới hạn, ID bằng 0.

Độ phủ chức năng của DMA gồm lớp độ dài (quanh một từ, một burst, 4 KB, rất dài), số burst của một descriptor, có vượt biên 4 KB hay không, độ lệch byte ở đầu và ở cuối, mã lỗi trong status và quan hệ giữa độ dài gói với descriptor. Mỗi engine được build với bốn cấu hình: mặc định (burst tối đa 16 nhịp, cho phép địa chỉ lệch), không cho phép địa chỉ lệch, burst tối đa 1 nhịp và burst tối đa 256 nhịp.




*Hình 6.3. Testbench của các engine DMA*

## 6.3. Các test đã thực hiện

*Bảng 6.1. Các test của ba engine DMA (số lần PASS trên tổng số lần chạy, gộp mọi cấu hình build)*

| Test | Nội dung kiểm tra | axi_dma_rd | axi_dma_wr | axi_cdma |
|---|---|---|---|---|
| dma_smoke_test | Một lần truyền ngắn và một lần truyền nhiều burst hoàn tất với dữ liệu và status đúng | 1/1 PASS | 1/1 PASS | 1/1 PASS |
| dma_random_test | Descriptor ngẫu nhiên thuộc mọi lớp độ dài và mọi độ lệch, nối đuôi nhau, dưới backpressure ngẫu nhiên | 24/24 PASS | 24/24 PASS | 24/24 PASS |
| dma_backpressure_test | Bộ nhớ và stream hiếm khi sẵn sàng làm các hàng đợi bên trong đầy lên; engine dừng và chạy tiếp đúng | 4/4 PASS | 4/4 PASS | 4/4 PASS |
| dma_throughput_test | Bốn tổ hợp bộ nhớ nhanh hoặc chậm với stream nhanh hoặc chậm | 3/3 PASS | 3/3 PASS | 3/3 PASS |

| Test | Nội dung kiểm tra | axi_dma_rd | axi_dma_wr | axi_cdma |
|---|---|---|---|---|
| dma_error_test | SLVERR và DECERR ở đầu, giữa hoặc cuối vùng truyền được báo trong status của đúng descriptor | 12/12 PASS | 12/12 PASS | 12/12 PASS |
| dma_reset_test | Reset ở thời điểm ngẫu nhiên, ở từng chu kỳ ngay sau khi nhận descriptor, và lúc đang chờ dữ liệu stream | 4/4 PASS | 4/4 PASS | 4/4 PASS |
| dma_enable_test | enable ở mức thấp thì không nhận descriptor; đổi enable ngẫu nhiên giữa lúc truyền không làm hỏng việc đang truyền | 3/3 PASS | 3/3 PASS | 3/3 PASS |
| dma_field_walk_test | Mọi bit của địa chỉ, tag, id, dest, user được truyền đúng tới AXI, stream và status | 1/1 PASS | 1/1 PASS | 1/1 PASS |
| dma_long_test | Hai lần truyền dài gần cực đại (2^20 − 1 byte), mọi byte đều được so sánh | 1/1 PASS | 1/1 PASS | 1/1 PASS |
| dma_resp_backlog_test | Giữ lại đáp ứng ghi: engine phải dừng đúng ở 32 burst chưa được trả lời rồi hoàn tất khi đáp ứng về | – | 3/3 PASS | 3/3 PASS |
| dma_cdma_offset_test | Mọi tổ hợp độ lệch byte của nguồn và đích khi dữ liệu đọc về liên tục | – | – | 3/3 PASS |
| dma_wr_short_frame_split_test | Gói ngắn hơn descriptor có nhịp cuối bị chia qua hai burst không làm ảnh hưởng gói kế tiếp | – | 3/3 PASS | – |
| dma_wr_abort_test | Ghi nhận hành vi của ngõ vào abort | – | 2/2 PASS | – |

## 6.4. Các lỗi tìm được ở DMA

– **CDMA-01** – axi_cdma làm sai dữ liệu khi độ lệch byte của nguồn và đích khác nhau và dữ liệu đọc về liên tục: ở chu kỳ nhận nhịp đọc cuối của một burst trong khi phía ghi còn nhịp phải xuất, RREADY bị giữ cao thêm một chu kỳ, nhịp đầu của burst đọc kế tiếp bị nhận rồi bỏ, từ đó dữ liệu ghi ra bị lệch.

– **DMAWR-01** – axi_dma_wr nuốt nhịp đầu của gói kế tiếp sau một gói ngắn có nhịp cuối bị chia qua hai burst: khi phát burst mới, TREADY được bật lại trong khi phần dư của nhịp cuối còn nằm trong thanh ghi dịch. Hậu quả là gói sau thiếu 4 byte và status mang id, dest, user sai.

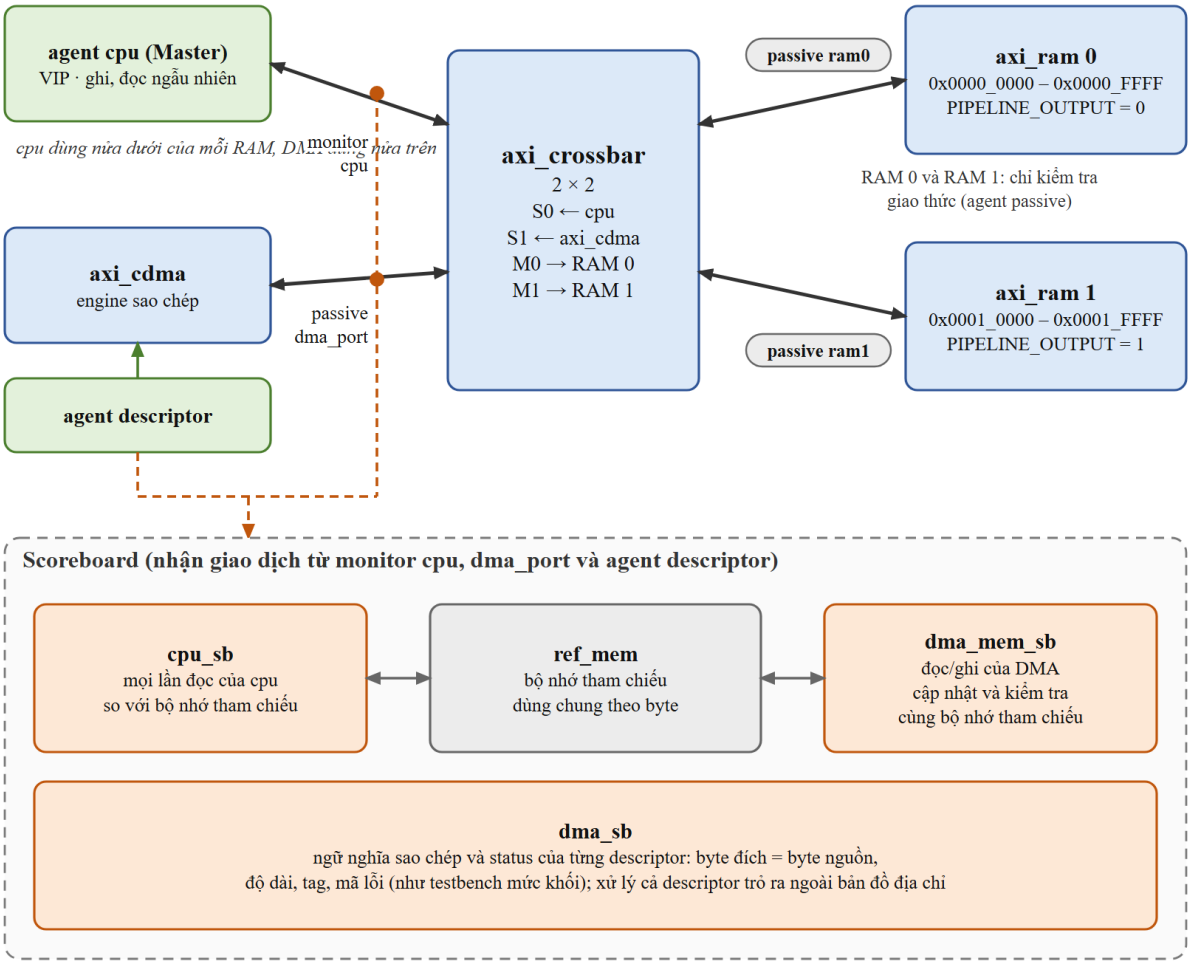
– **DMA-01** – Ngõ vào abort của axi_dma_wr được khai báo nhưng không được dùng; kéo abort trong lúc truyền không có tác dụng. Đây là một giới hạn, mã gốc cũng như vậy.

Hai lỗi đầu có bản sửa tuỳ chọn chỉ một dòng mỗi lỗi. Để xác định lỗi thuộc về mã gốc hay do phần chỉnh sửa trong project gây ra, bản gốc nguyên vẹn của các file DMA đã được tải về một thư mục riêng và chạy với đúng testbench này: cả hai lỗi đều tái hiện trên mã gốc với cùng triệu chứng. Kết quả chạy toàn bộ regression trên mã gốc được trình bày ở mục 7.2.

## 6.5. Hệ thống tích hợp

Testbench hệ thống ghép các khối lại như trong một SoC nhỏ: một agent master của VIP đóng vai bộ xử lý và một axi_cdma cùng là master của một axi_crossbar 2 × 2; hai axi_ram 64 KB là slave, một RAM có PIPELINE_OUTPUT = 1. Các agent passive của VIP được gắn vào cổng của DMA và cổng của hai RAM để kiểm tra giao thức trên các bus nội bộ.

Hai scoreboard bộ nhớ, một cho bộ xử lý và một cho DMA, dùng chung một bộ nhớ tham chiếu: những gì một master ghi thì master kia phải đọc được. Scoreboard DMA kiểm tra ngữ nghĩa sao chép và status như ở testbench mức khối.




*Hình 6.4. Testbench hệ thống tích hợp*

*Bảng 6.2. Các test của hệ thống tích hợp*

| Test | Nội dung kiểm tra | Số lần chạy | Kết quả |
|---|---|---|---|
| sys_smoke_test | Bộ xử lý ghi vào một RAM, DMA chép sang RAM kia, bộ xử lý đọc lại đúng | 1 | 1/1 PASS |
| sys_random_test | Bộ xử lý và DMA dùng đồng thời cả hai RAM qua bộ kết nối mà không ảnh hưởng lẫn nhau | 6 | 6/6 PASS |
| sys_decerr_test | Descriptor trỏ ra ngoài bản đồ địa chỉ kết thúc bằng status DECERR; hệ thống tiếp tục hoạt động (phát từng descriptor một) | 4 | 4/4 PASS |
| sys_decerr_pipelined_test | Descriptor lỗi nối đuôi ngay sau descriptor hợp lệ: mã lỗi phải nằm trong đúng status | 1 | XFAIL (XBAR-03) |

Testbench hệ thống cho thấy hậu quả thực tế của XBAR-03. axi_cdma chỉ dùng ID 0 và phát địa chỉ ghi của descriptor kế tiếp trước khi nhận xong đáp ứng của descriptor hiện tại. Nếu descriptor kế tiếp ghi ra ngoài bản đồ địa chỉ, đáp ứng DECERR của bộ

kết nối vượt lên trước đáp ứng của RAM, và engine gán lỗi cho descriptor hợp lệ trong khi descriptor sai lại được báo thành công. Test cuối trong bảng tái hiện đúng tình huống này và được đánh dấu XFAIL.

# 7. TỔNG KẾT

## 7.1. Kết quả regression

*Bảng 7.1. Kết quả regression theo testbench*

| Testbench | Số lần chạy | PASS | FAIL | XFAIL | Số phép so sánh (triệu) |
|---|---|---|---|---|---|
| VIP tự kiểm (vip) | 23 | 23 | 0 | 0 | 0,80 |
| axi_ram (ram) | 44 | 43 | 0 | 1 | 1,73 |
| axi_crossbar (xbar) | 79 | 76 | 0 | 3 | 0,04 |
| axi_dma_rd (dma_rd) | 53 | 53 | 0 | 0 | 2,96 |
| axi_dma_wr (dma_wr) | 61 | 61 | 0 | 0 | 2,84 |
| axi_cdma (cdma) | 59 | 59 | 0 | 0 | 3,01 |
| Hệ thống (sys) | 12 | 11 | 0 | 1 | 1,21 |
| Các build chỉ để minh hoạ lỗi | 9 | 0 | 0 | 9 | 0,00 |
| Tổng cộng | 340 | 326 | 0 | 14 | 12,60 |

Không có lần chạy nào FAIL hoặc XPASS. 14 lần XFAIL là các test minh hoạ lỗi và giới hạn của thiết kế, gồm cả các lần chạy trên RTL gốc, trên các bản chỉ sửa một phần và trên file gốc nguyên vẹn của thư viện. 283 lần chạy tham chiếu trên mã gốc (mục 7.2) không được tính vào các con số của bảng này.

## 7.2. Chạy toàn bộ test trên mã gốc nguyên bản

Các con số ở mục 7.1 và 7.3 được đo trên RTL đã bật bảy bản sửa tuỳ chọn và, với bốn file đã bị chỉnh sửa trong project, trên bản của project. Để biết mã gốc của thư viện qua được bao nhiêu test, toàn bộ test chính được chạy lại, với đúng các seed và cấu hình build đó, trên RTL không có bản sửa nào: bộ kết nối build không định nghĩa macro sửa lỗi nào (tám file giống hệt mã gốc), còn axi_ram, axi_dma_rd, axi_dma_wr, axi_cdma lấy nguyên vẹn từ kho của tác giả [6]. Riêng axi_ram gốc không có tham số INIT_FILE nên cấu hình init và test tương ứng không áp dụng. Các test minh hoạ lỗi (XFAIL) đã được chạy riêng trên mã gốc nên không lặp lại. Kết quả được ghi là ORIG-PASS hoặc ORIG-FAIL và không bao giờ được tính là lỗi của regression.

*Bảng 7.2. Kết quả chạy các test chính trên mã gốc nguyên bản*

| Thiết kế (mã gốc) | Số lần chạy | Qua | Không qua | Tỷ lệ qua |
|---|---|---|---|---|
| axi_ram gốc | 43 | 42 | 1 | 97,7% |
| axi_crossbar không bật bản sửa nào | 73 | 5 | 68 | 6,8% |

| Thiết kế (mã gốc) | Số lần chạy | Qua | Không qua | Tỷ lệ qua |
|---|---|---|---|---|
| axi_dma_rd gốc | 53 | 53 | 0 | 100,0% |
| axi_dma_wr gốc | 58 | 49 | 9 | 84,5% |
| axi_cdma gốc | 56 | 14 | 42 | 25,0% |
| Tổng cộng | 283 | 163 | 120 | 57,6% |

Bảng dưới liệt kê từng test có ít nhất một lần không qua; cột cuối là các dấu hiệu tìm thấy trong log của những lần chạy đó (tìm bằng các mẫu lỗi đặc trưng, không phải kết luận chắc chắn về nguyên nhân gốc – nguyên nhân gốc của từng lỗi đã được phân tích riêng ở các mục 5 và 6).

*Bảng 7.3. Các test không qua trên mã gốc*

| Thiết kế | Test | Cấu hình | Không qua / tổng | Dấu hiệu trong log |
|---|---|---|---|---|
| axi_cdma | dma_backpressure_test | default | 4 / 4 | treo / hết thời gian chờ; dữ liệu đích khác dữ liệu nguồn (CDMA-01) |
| axi_cdma | dma_enable_test | default | 3 / 3 | treo / hết thời gian chờ; dữ liệu đích khác dữ liệu nguồn (CDMA-01) |
| axi_cdma | dma_error_test | burst256 | 2 / 2 | treo / hết thời gian chờ; dữ liệu đích khác dữ liệu nguồn (CDMA-01) |
| axi_cdma | dma_error_test | default | 6 / 6 | treo / hết thời gian chờ; dữ liệu đích khác dữ liệu nguồn (CDMA-01) |
| axi_cdma | dma_long_test | default | 1 / 1 | treo / hết thời gian chờ; dữ liệu đích khác dữ liệu nguồn (CDMA-01) |
| axi_cdma | dma_random_test | burst256 | 4 / 4 | treo / hết thời gian chờ; dữ liệu đích khác dữ liệu nguồn (CDMA-01) |
| axi_cdma | dma_random_test | default | 12 / 12 | treo / hết thời gian chờ; dữ liệu đích khác dữ liệu nguồn (CDMA-01) |
| axi_cdma | dma_reset_test | default | 4 / 4 | treo / hết thời gian chờ; dữ liệu đích khác dữ liệu nguồn (CDMA-01) |
| axi_cdma | dma_resp_backlog_test | default | 3 / 3 | treo / hết thời gian chờ; dữ liệu đích khác dữ liệu nguồn (CDMA-01) |
| axi_cdma | dma_throughput_test | default | 3 / 3 | treo / hết thời gian chờ; dữ liệu đích khác dữ liệu nguồn (CDMA-01) |
| axi_dma_wr | dma_enable_test | default | 2 / 3 | sai status hoặc thiếu byte ghi (DMAWR-01); treo / hết thời gian chờ |
| axi_dma_wr | dma_error_test | burst1 | 1 / 2 | treo / hết thời gian chờ; sai status hoặc thiếu byte ghi (DMAWR-01) |
| axi_dma_wr | dma_random_test | burst1 | 4 / 4 | treo / hết thời gian chờ; sai status hoặc thiếu byte ghi (DMAWR-01) |

| Thiết kế | Test | Cấu hình | Không qua / tổng | Dấu hiệu trong log |
|---|---|---|---|---|
| axi_dma_wr | dma_random_test | default | 2 / 12 | sai status hoặc thiếu byte ghi (DMAWR-01); treo / hết thời gian chờ |
| axi_ram | axi_ram_wrap_test | default | 1 / 1 | dữ liệu sai khi burst WRAP (RAM-01) |
| axi_crossbar (không sửa) | xbar_backpressure_test | default | 4 / 4 | đáp ứng không khớp giao dịch nào; treo / hết thời gian chờ; mất nhịp ghi sau DECERR (XBAR-05) |
| axi_crossbar (không sửa) | xbar_backpressure_test | open | 2 / 2 | mất nhịp ghi sau DECERR (XBAR-05); đáp ứng không khớp giao dịch nào; treo / hết thời gian chờ |
| axi_crossbar (không sửa) | xbar_decerr_test | default | 4 / 4 | đáp ứng không khớp giao dịch nào; treo / hết thời gian chờ; mất nhịp ghi sau DECERR (XBAR-05) |
| axi_crossbar (không sửa) | xbar_decerr_wdata_test | default | 3 / 3 | mất nhịp ghi sau DECERR (XBAR-05); treo / hết thời gian chờ |
| axi_crossbar (không sửa) | xbar_directed_test | default | 2 / 2 | treo / hết thời gian chờ |
| axi_crossbar (không sửa) | xbar_directed_test | open | 1 / 1 | treo / hết thời gian chờ |
| axi_crossbar (không sửa) | xbar_id_thread_test | default | 6 / 6 | treo / hết thời gian chờ; vượt giới hạn M_ISSUE (XBAR-08) |
| axi_crossbar (không sửa) | xbar_id_thread_test | open | 3 / 3 | vượt giới hạn M_ISSUE (XBAR-08) |
| axi_crossbar (không sửa) | xbar_issue_limit_test | default | 3 / 3 | vượt giới hạn M_ISSUE (XBAR-08); treo / hết thời gian chờ |
| axi_crossbar (không sửa) | xbar_issue_limit_test | open | 3 / 3 | vượt giới hạn M_ISSUE (XBAR-08) |
| axi_crossbar (không sửa) | xbar_param_test | open | 1 / 1 | tham số register slice không được truyền (XBAR-06) |
| axi_crossbar (không sửa) | xbar_random_test | default | 12 / 12 | mất nhịp ghi sau DECERR (XBAR-05); đáp ứng không khớp giao dịch nào; treo / hết thời gian chờ; vượt giới hạn M_ISSUE (XBAR-08) |
| axi_crossbar (không sửa) | xbar_random_test | open | 6 / 6 | vượt giới hạn M_ISSUE (XBAR-08); mất nhịp ghi sau DECERR (XBAR-05); đáp ứng không khớp giao dịch nào; treo / hết thời gian chờ |
| axi_crossbar (không sửa) | xbar_reset_drop_test | default | 3 / 3 | treo / hết thời gian chờ |

| Thiết kế | Test | Cấu hình | Không qua / tổng | Dấu hiệu trong log |
|---|---|---|---|---|
| axi_crossbar (không sửa) | xbar_reset_test | default | 4 / 4 | đáp ứng không khớp giao dịch nào; treo / hết thời gian chờ |
| axi_crossbar (không sửa) | xbar_slave_error_test | default | 4 / 4 | mất nhịp ghi sau DECERR (XBAR-05); đáp ứng không khớp giao dịch nào; treo / hết thời gian chờ; vượt giới hạn M_ISSUE (XBAR-08) |
| axi_crossbar (không sửa) | xbar_slave_error_test | open | 2 / 2 | vượt giới hạn M_ISSUE (XBAR-08) |
| axi_crossbar (không sửa) | xbar_slave_skew_test | default | 3 / 3 | treo / hết thời gian chờ |
| axi_crossbar (không sửa) | xbar_slave_skew_test | open | 2 / 2 | vượt giới hạn M_ISSUE (XBAR-08) |

- **axi_ram gốc**: 42/43 lần chạy qua; lần không qua duy nhất là test burst WRAP (RAM-01, giới hạn đã công bố). Việc thêm tham số INIT_FILE không làm thay đổi hành vi được kiểm tra.
- **axi_dma_rd gốc**: 53/53 lần chạy qua, tức là đọc DMA gốc và bản đã chỉnh sửa trong project cho cùng kết quả trên toàn bộ bộ test.
- **axi_dma_wr gốc**: 49/58 lần chạy qua. Trong 9 lần không qua có 9 lần có dấu hiệu của DMAWR-01 (sai status hoặc thiếu byte ghi).
- **axi_cdma gốc**: chỉ 14/56 lần chạy qua. Trong 42 lần không qua có 40 lần dữ liệu đích khác dữ liệu nguồn (CDMA-01); các lần còn lại chỉ có dấu hiệu treo hoặc hết thời gian chờ. Lỗi xuất hiện ở hầu hết các test vì các descriptor ngẫu nhiên thường có độ lệch byte của nguồn và đích khác nhau, nên ngay cả test ngẫu nhiên thông thường cũng gặp.
- **axi_crossbar không bật bản sửa nào**: chỉ 5/73 lần chạy qua. Trong 68 lần không qua có 56 lần treo hoặc hết thời gian chờ, 17 lần mất nhịp ghi sau DECERR (XBAR-05), 19 lần vượt giới hạn M_ISSUE (XBAR-08) và 33 lần đáp ứng không khớp giao dịch nào. Hầu hết test của bộ kết nối có chứa truy cập sai địa chỉ (DECERR), nên dấu hiệu treo phù hợp với lỗi XBAR-02 đã được chứng minh riêng bằng test xbar_decerr_lockup_test; bộ kiểm tra giới hạn đồng thời cũng bắt được XBAR-08 ở nhiều test có tải đủ lớn.

Hệ quả thực tế là: nếu dùng mã gốc, hầu như không thể chạy một test ngẫu nhiên có truy cập sai địa chỉ trên bộ kết nối, và không thể chạy bộ test ngẫu nhiên của axi_cdma. Chính vì vậy regression chính được chạy trên RTL đã bật các bản sửa tuỳ chọn: nếu không, các lỗi đã biết sẽ che mất mọi vấn đề khác và độ phủ không đo được.

## 7.3. Độ phủ mã

Độ phủ mã được đo trên RTL với sáu loại: câu lệnh, nhánh, điều kiện, biểu thức, máy trạng thái và chuyển mức tín hiệu. Con số tổng là giá trị Total Coverage do QuestaSim tính, bằng trung bình của các loại trên. Để thấy rõ ảnh hưởng của việc loại trừ, mỗi nhóm được báo cáo bằng ba con số.

*Bảng 7.4. Độ phủ mã tổng theo nhóm (%)*

| Nhóm | Không loại trừ | Chỉ loại trừ mục không thể đạt | Loại trừ thêm mục giới hạn bởi cấu hình |
|---|---|---|---|
| RAM | 94,95 | 100,00 | 100,00 |
| Interconnect | 94,39 | 99,77 | 100,00 |
| DMA | 96,03 | 98,84 | 100,00 |

*Bảng 7.5. Độ phủ mã của nhóm RAM theo từng loại*

| Loại | Không loại trừ | Chỉ loại trừ mục không thể đạt | Loại trừ thêm mục giới hạn bởi cấu hình |
|---|---|---|---|
| Câu lệnh | 100,00% (143/143) | 100,00% (143/143) | 100,00% (143/143) |
| Nhánh | 95,12% (39/41) | 100,00% (39/39) | 100,00% (39/39) |
| Điều kiện | 91,30% (21/23) | 100,00% (23/23) | 100,00% (23/23) |
| Biểu thức | 100,00% (6/6) | 100,00% (6/6) | 100,00% (6/6) |
| Trạng thái FSM | 100,00% (3/3) | 100,00% (3/3) | 100,00% (3/3) |
| Chuyển trạng thái FSM | 100,00% (4/4) | 100,00% (4/4) | 100,00% (4/4) |
| Chuyển mức tín hiệu | 83,33% (1030/1236) | 100,00% (1028/1028) | 100,00% (1028/1028) |
| Tổng | 94,95% | 100,00% | 100,00% |

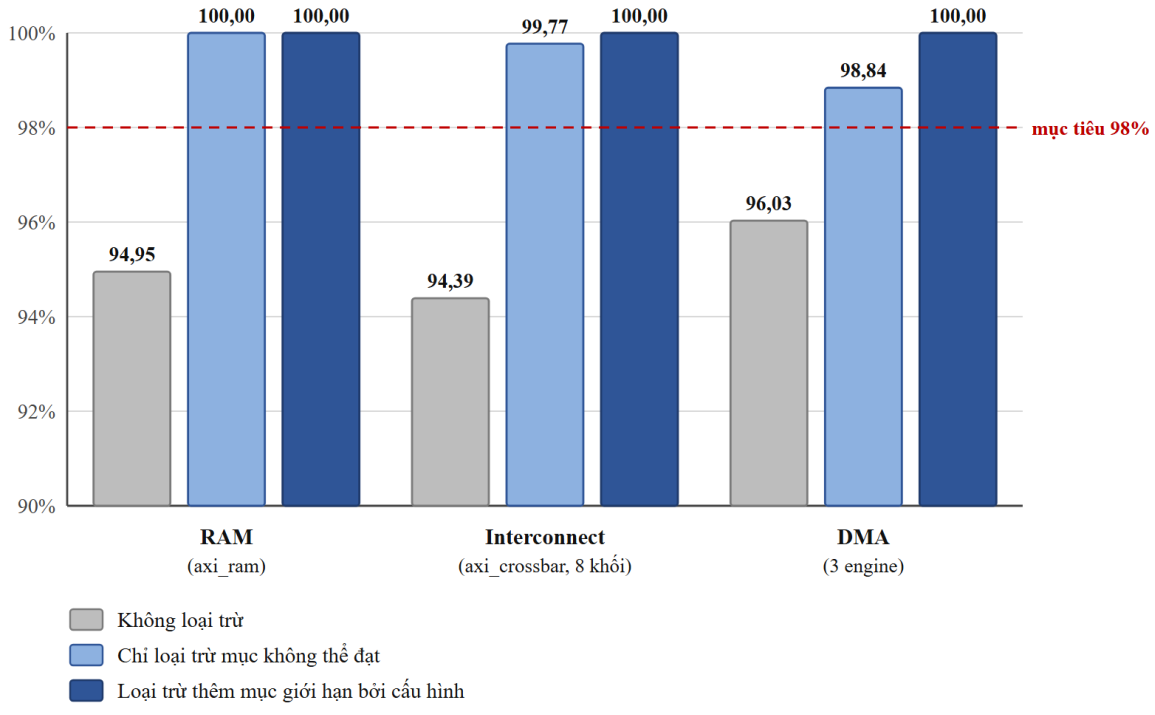
*Bảng 7.6. Độ phủ mã của nhóm Interconnect theo từng loại*

| Loại | Không loại trừ | Chỉ loại trừ mục không thể đạt | Loại trừ thêm mục giới hạn bởi cấu hình |
|---|---|---|---|
| Câu lệnh | 97,51% (978/1003) | 100,00% (978/978) | 100,00% (978/978) |
| Nhánh | 95,76% (226/236) | 100,00% (221/221) | 100,00% (221/221) |
| Điều kiện | 86,05% (74/86) | 100,00% (78/78) | 100,00% (78/78) |
| Biểu thức | 92,44% (318/344) | 100,00% (344/344) | 100,00% (344/344) |

| Loại | Không loại trừ | Chỉ loại trừ mục không thể đạt | Loại trừ thêm mục giới hạn bởi cấu hình |
|---|---|---|---|
| Trạng thái FSM | 100,00% (2/2) | 100,00% (2/2) | 100,00% (2/2) |
| Chuyển trạng thái FSM | 100,00% (2/2) | 100,00% (2/2) | 100,00% (2/2) |
| Chuyển mức tín hiệu | 94,59% (6500/6872) | 98,66% (6496/6584) | 100,00% (6496/6496) |
| Tổng | 94,39% | 99,77% | 100,00% |

*Bảng 7.7. Độ phủ mã của nhóm DMA theo từng loại*

| Loại | Không loại trừ | Chỉ loại trừ mục không thể đạt | Loại trừ thêm mục giới hạn bởi cấu hình |
|---|---|---|---|
| Câu lệnh | 100,00% (934/934) | 100,00% (934/934) | 100,00% (934/934) |
| Nhánh | 96,05% (219/228) | 100,00% (219/219) | 100,00% (219/219) |
| Điều kiện | 92,31% (108/117) | 96,58% (113/117) | 100,00% (117/117) |
| Biểu thức | 95,28% (101/106) | 100,00% (106/106) | 100,00% (106/106) |
| Trạng thái FSM | 100,00% (10/10) | 100,00% (10/10) | 100,00% (10/10) |
| Chuyển trạng thái FSM | 100,00% (19/19) | 100,00% (19/19) | 100,00% (19/19) |
| Chuyển mức tín hiệu | 92,57% (6254/6756) | 96,48% (6252/6480) | 100,00% (6252/6252) |
| Tổng | 96,03% | 98,84% | 100,00% |




*Lưu ý: trục dọc cắt từ 90%. Độ phủ mã tổng = trung bình 6 loại (câu lệnh, nhánh, điều kiện, biểu thức, FSM, chuyển mức).*

*Hình 7.1. Độ phủ mã trước và sau loại trừ*

## 7.4. Chính sách loại trừ

Tổng cộng có 271 lệnh loại trừ trong ba file (dma.do: 184 lệnh; ram.do: 13 lệnh; xbar.do: 74 lệnh), mỗi lệnh kèm lý do. Quy tắc áp dụng: gặp một lỗ độ phủ thì trước hết tìm kích thích hợp lệ có thể chạm tới và viết test; chỉ khi chứng minh được bằng RTL là không thể đạt thì mới loại trừ.

- **Không thể đạt trong mọi cấu hình**: biến vòng lặp kiểu integer (không phải phần cứng); tín hiệu gán hằng; mã chỉ chạy lúc elaboration; các kiểm tra tham số kết thúc mô phỏng; nhánh mặc định của lệnh case đã liệt kê đủ trạng thái; các hàng điều kiện mâu thuẫn về cấu trúc.
- **Giới hạn bởi cấu hình đã kiểm chứng** (162 lệnh): các bit cao của bộ đếm byte và bộ đếm nhịp không bao giờ dùng tới với bus dữ liệu 32 bit; bit cao của AxREGION khi mỗi slave chỉ có ba vùng địa chỉ. Các mục này không phải không thể đạt về nguyên tắc, nên được báo cáo thành một cột riêng.

Một số lỗ độ phủ đã được đóng bằng test mới thay vì loại trừ: giữ đáp ứng ghi để chạm giới hạn 32 burst của engine DMA, reset đúng chu kỳ ngay sau khi nhận descriptor, phát TLAST ngẫu nhiên khi TVALID ở mức thấp, hoán vị giới hạn M_ISSUE giữa hai cấu hình build, reset đúng lúc bộ kết nối đang bỏ dữ liệu của một lần ghi bị từ chối.

## 7.5. Độ phủ chức năng

*Bảng 7.8. Độ phủ chức năng (covergroup) theo testbench*

| Testbench | Số lần chạy được gộp | Số kiểu covergroup | Độ phủ (%) |
|---|---|---|---|
| VIP tự kiểm | 23 | 17 | 100,00 |
| axi_ram | 43 | 13 | 100,00 |
| axi_crossbar | 76 | 20 | 100,00 |
| axi_dma_rd | 53 | 7 | 100,00 |
| axi_dma_wr | 61 | 9 | 100,00 |
| axi_cdma | 59 | 9 | 100,00 |
| Hệ thống (tham khảo) | 11 | 16 | 91,77 |

Testbench hệ thống dùng để kiểm tra việc ghép nối; độ phủ của nó chỉ để tham khảo và không được cộng vào số liệu mức khối (độ phủ mã tổng mà riêng testbench hệ thống đạt được: RAM 83,44%; Interconnect 85,52%; DMA 83,59%).

## 7.6. Tổng hợp các lỗi và giới hạn của thiết kế

*Bảng 7.9. Mười hai lỗi và giới hạn do môi trường kiểm chứng tìm ra*

| Mã | Khối | Tóm tắt | Mức độ chắc chắn | Bản sửa tuỳ chọn |
|---|---|---|---|---|
| RAM-01 | axi_ram | Burst WRAP bị xử lý như INCR | Giới hạn đã được công bố trong tài liệu của thư viện gốc | Không |
| XBAR-01 | crossbar | Đáp ứng DECERR phát trước khi yêu cầu được nhận xong | Lỗi giao thức; đã có người khác báo trên kho mã gốc | Không |
| XBAR-02 | crossbar | Master bị khoá sau DECERR | Lỗi; đã có người khác báo trên kho mã gốc | Có |
| XBAR-03 | crossbar | DECERR sai thứ tự so với đáp ứng cùng ID; ở mức hệ thống DMA báo lỗi nhầm descriptor | Chỉ đồ án này tìm ra; chưa có xác nhận độc lập | Không |
| XBAR-04 | crossbar | Slave xen kẽ dữ liệu đọc làm nhịp tới sai master | Tính năng chưa hỗ trợ, tác giả đã ghi nhận | Không |
| XBAR-05 | crossbar | Mất nhịp ghi đầu tiên sau một lần ghi bị DECERR | Chỉ đồ án này tìm ra; chưa có xác nhận độc lập | Có |
| XBAR-06 | crossbar | Tham số kiểu register slice phía master không được truyền xuống | Lỗi; đã có người khác báo trên kho mã gốc | Có |
| XBAR-07 | crossbar | Đáp ứng tới sai master khi kênh đáp ứng phía master có thanh ghi | Chỉ đồ án này tìm ra; chưa có xác nhận độc lập | Có |
| XBAR-08 | crossbar | Giới hạn M_ISSUE bị vượt; với giới hạn bằng 1 thì mất hẳn | Chỉ đồ án này tìm ra; chưa có xác nhận độc lập | Có |
| CDMA-01 | axi_cdma | Mất một nhịp đọc khi nguồn và đích lệch byte khác nhau | Tái hiện được trên file gốc nguyên vẹn; chưa có xác nhận độc lập | Có |
| DMAWR-01 | axi_dma_wr | Nuốt nhịp đầu của gói kế tiếp sau một gói ngắn bị chia | Tái hiện được trên file gốc nguyên vẹn; chưa có xác nhận độc lập | Có |
| DMA-01 | axi_dma_wr | Ngõ vào abort không có tác dụng | Giới hạn; mã gốc cũng như vậy | Không |

Với các lỗi mà chỉ đồ án này tìm ra, bằng chứng gồm: một test tự kiểm tra thất bại trên RTL chưa sửa, nguyên nhân chỉ ra được ở dòng RTL cụ thể, và bản sửa tương ứng làm chính test đó PASS cùng toàn bộ regression. Kho mã gốc đã ngừng bảo trì và được thay bằng một thư viện kế nhiệm; các lỗi nằm ở những góc ít được dùng tới như truy

cập sai địa chỉ, tham số khác mặc định hoặc hai master tranh chấp một slave có giới hạn nhỏ.

## 7.7. Đối chiếu với mục tiêu

- VIP AXI4-Full dùng lại được: đạt. Cùng một bộ mã nguồn được dùng ở cả năm testbench với ba vai trò master, slave và passive, và đã tự kiểm chứng kể cả bằng test âm.
- Kiểm chứng năm khối RTL và hệ thống tích hợp bằng test tự kiểm tra, phần lớn là ngẫu nhiên nhiều seed: đạt.
- Độ phủ tổng trên 98% cho từng nhóm: đạt sau khi loại trừ có lý do (RAM 100,00%, interconnect 99,77%, DMA 98,84% khi chỉ loại trừ các mục không thể đạt). Số chưa loại trừ là 94,95%, 94,39% và 96,03%.
- Log của mỗi test có đủ mục đích, kích thích, cách kiểm tra, cấu hình, từng giao dịch và thống kê: đạt.

## 7.8. Hạn chế

- Số liệu độ phủ được đo trên RTL đã bật bảy bản sửa tuỳ chọn; nếu không bật thì nhiều test ngẫu nhiên vấp cùng một lỗi và không đo được phần còn lại. Việc có đưa các bản sửa vào thiết kế hay không thuộc về người thiết kế.
- Độ phủ mã chưa được đo trên mã gốc nguyên bản: các lần chạy trên mã gốc (mục 7.2) chỉ ghi nhận qua hoặc không qua, và chỉ nhờ nhật ký mới phân loại được dấu hiệu lỗi.
- Chỉ kiểm chứng bus dữ liệu 32 bit và một hình học của bộ kết nối (2 master, 3 slave) với hai biến thể kết nối.
- Ba vấn đề của bộ kết nối chưa có bản sửa (XBAR-01, XBAR-03, XBAR-04); các test thông thường phải tránh hoặc miễn trừ có đếm những tình huống này.
- Giới hạn S_ACCEPT chỉ kiểm tra được ở cổng với biên nới theo độ sâu của register slice; S_THREADS không được kiểm tra trực tiếp.
- Độ phủ được thu với thiết lập mặc định của trình mô phỏng; tuỳ chọn khử glitch chặt hơn làm phiên bản QuestaSim 10.7c bị lỗi nên không dùng được.
- VIP chưa hỗ trợ truy cập độc quyền, AXI3 và AXI4-Lite.

## 7.9. Hướng phát triển

- Mở rộng kiểm chứng sang các độ rộng dữ liệu 64 và 128 bit để đóng các mục hiện đang bị giới hạn bởi cấu hình.
- Thêm các hình học khác của bộ kết nối (nhiều master, nhiều slave hơn) và kiểm tra trực tiếp S_THREADS.
- Bổ sung cho VIP mô hình truy cập độc quyền, chế độ AXI4-Lite và khả năng tiêm lỗi giao thức ở phía master.
- Kiểm chứng hình thức (formal) cho các tính chất giao thức của bộ kết nối, nhất là thứ tự đáp ứng theo ID.
- Áp dụng môi trường này cho thư viện kế nhiệm của mã gốc để kiểm tra xem các lỗi đã được khắc phục hay chưa.

## 7.10. Kết luận

Đồ án đã xây dựng được một VIP AXI4-Full dùng lại được và một môi trường kiểm chứng UVM hoàn chỉnh cho bộ nhớ, bộ kết nối và ba engine DMA, từ mức khối tới mức hệ thống. Môi trường tự kiểm tra hoàn toàn, chạy 340 lần mô phỏng không có lần nào thất bại ngoài dự kiến, đạt độ phủ chức năng 100% ở mức khối và độ phủ mã trên 98% sau khi loại trừ có lý do. Quan trọng hơn các con số, môi trường đã chứng tỏ giá trị của mình bằng việc tìm ra mười hai lỗi và giới hạn trong một thư viện mã nguồn mở được dùng rộng rãi, trong đó có những lỗi gây treo, mất dữ liệu và trả lời nhầm master; vài lỗi chỉ lộ ra khi phân tích kỹ các lỗ độ phủ thay vì vội loại trừ chúng.

# TÀI LIỆU THAM KHẢO


[1] Arm Limited, “AMBA AXI and ACE Protocol Specification”, tài liệu Arm IHI 0022 (bản mới nhất tại thời điểm thực hiện đồ án), mô tả AXI3, AXI4, AXI4-Lite.

[2] Arm Limited, “AMBA 4 AXI4-Stream Protocol Specification”, tài liệu Arm IHI 0051.

[3] Accellera Systems Initiative, “Universal Verification Methodology (UVM) 1.2 User’s Guide”, 2015.

[4] IEEE Std 1800.2-2017, “IEEE Standard for Universal Verification Methodology Language Reference Manual”.

[5] IEEE Std 1800-2017, “IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language”.

[6] A. Forencich, “verilog-axi: Verilog AXI, AXI lite, and AXI stream components for FPGA implementation”, kho mã nguồn GitHub, https://github.com/alexforencich/verilog-axi (nhánh master, mã nguồn và README được đối chiếu ngày 12/10/2026). Kho này đã ngừng bảo trì và được thay bằng https://github.com/fpganinja/taxi.

[7] Các issue của kho verilog-axi: #30 “Support read data interleaving”, #79 “Parameters not passed on to axi_crossbar_wr and axi_crossbar_rd in axi_crossbar.v”, #84 “Crossbar locks up after reading to write-only address”, #95 “CROSSBAR: AXI write DECERR response assert BVALID before WLAST on multi-beat burst”, https://github.com/alexforencich/verilog-axi/issues (truy cập ngày 12/10/2026).

[8] Mentor Graphics (Siemens EDA), “Questa SIM User’s Manual” và “Questa SIM Command Reference Manual”, phiên bản 10.7c, 2018.

[9] C. Spear, G. Tumbush, “SystemVerilog for Verification: A Guide to Learning the Testbench Language Features”, ấn bản lần 3, Springer, 2012.